

NÃO ESTÁ PROVADO

*Como um autodidata aos 40 anos aprendeu a
construir software real com IA — da primeira
conversa a uma operação de agentes*

Evandro Henn

NÃO ESTÁ PROVADO

Como um autodidata aos 40 anos aprendeu a construir software real com IA — da primeira conversa a uma operação de agentes

Copyright © 2026 Evandro Henn. Todos os direitos reservados.

Nenhuma parte deste livro pode ser reproduzida ou transmitida, por qualquer meio, sem autorização escrita do autor, exceto citações breves com indicação da fonte.

Primeira edição: 2026.

ISBN: a definir.

Os contratos evidence-contracts e work-order-contract, citados no livro, estão em domínio público (CC0). Este livro não está.

Os nomes de produtos, serviços e empresas citados pertencem a seus respectivos donos e aparecem apenas para descrever fatos.

A narrativa para em 22 de setembro de 2026.

Contato: contato@edgebuildlabs.tech · edgebuildlabs.tech

Sumário

Prólogo — A máquina estava pronta. E agora?	1
Prelúdio — O código não foi o começo	4
PARTE I — PRESERVAR	14
1. A memória estava cheia	15
2. Inteligência no núcleo, ruído na transmissão	18
3. Quando uma conversa virou arquitetura	21
PARTE II — GOVERNAR	23
4. Instituições ao redor da inteligência	24
5. Production ready: no	27
PARTE III — SERVIR	29
6. O primeiro sistema que alguém usou	30
7. Quando o código pode agir contra sua intenção	33
PARTE IV — JULGAR	36
8. Transformar estratégia em ensino	37
9. Cento e vinte e sete testes e um não	39
10. Trinta e oito cenários para voltar ao começo	42
PARTE V — MULTIPLICAR	44
11. Vinte dias	45
12. MegaBrain não era uma ferramenta	48
13. Model said done	51
14. Produto PASS, processo PARTIAL	54
15. O benchmark que não aconteceu	56
16. Máxima autonomia dentro, mínima superfície fora	59
PARTE VI — ASSUMIR	62
17. O que um desenvolvedor júnior faz agora?	63
Ponto de parada — 18 de agosto de 2026	69
PARTE VII — PRODUZIR	71
18. A oficina se refez	72
19. O mundo ficou público	76
20. Cinco produtos, receita zero	80
21. Ruído na transmissão, de novo	84
22. A quarta pasta	88

23. Um vídeo sobre o que estava acontecendo	93
Epílogo — O objeto mudou	98
Nota sobre o método	102
Apêndice — Vinte e três regras para construir com agentes sem se enganar	104
Glossário	107
Notas documentais	111
Sobre o autor	114

Prólogo — A máquina estava pronta. E agora?

Na madrugada de 18 de agosto de 2026, eu estava diante de uma coisa que, três meses antes, não saberia sequer descrever.

Eu tinha 40 anos e nunca havia trabalhado como programador. Minha vida profissional tinha passado por um concurso público, por atas de sessões legislativas e por aproximadamente duas décadas de poker online. O registro mais antigo que consegui recuperar do meu uso de inteligência artificial é de maio de 2026: eu a usava para organizar o que sabia de poker. Em poucas semanas, passei a usá-la para construir coisas que nunca tinha sido capaz de construir sozinho.

Três meses depois, eu tinha montado um sistema que colocava vários programas de inteligência artificial para trabalhar juntos. Não programas que apenas conversam, mas agentes: programas que recebem uma tarefa e a executam — abrem arquivos, rodam comandos, escrevem código, testam o que escreveram. Um agente construía. Outro criticava o que o primeiro havia feito. Outro testava. A maior parte rodava num servidor alugado, longe do meu computador pessoal, dentro de compartimentos fechados para que nenhum deles alcançasse o que não devia.

Eu chamei aquilo de MegaBrain.

Havia também uma quantidade absurda de documentos. Versões, relatórios, registros de cada execução, decisões tomadas e depois superadas, falhas guardadas de propósito para não serem esquecidas. Instruções escritas para impedir que a instrução seguinte apagasse o que a anterior havia decidido.

Naquela madrugada, a última peça tinha sido fechada. “Fechada” não queria dizer perfeita. Queria dizer que a pergunta que justificava aquela fase havia sido respondida: as peças funcionavam, cada uma dentro dos seus limites, e o sistema sabia distinguir uma tarefa começada de uma tarefa realmente terminada.

O registro daquela noite terminava com uma linha curta, em inglês técnico:

NEXT = PRODUCTIZATION / REAL PROJECT VALUE

Em português: o próximo passo era transformar aquilo em produto e em valor de verdade.

E então apareceu a pergunta mais desconfortável de todas:

o que, exatamente, aquilo ia produzir?

Até ali, eu havia passado dias provando capacidades. Um agente conseguia editar arquivos. Outro conseguia revisar. O trabalho sobrevivia a uma reinicialização do servidor. Um agente crítico encontrava o caso raro que o agente construtor não tinha previsto. Tudo isso importava.

Mas capacidade não é consequência.

Uma oficina impecável não prova que o móvel será bom. Um laboratório equipado não prova que a pesquisa vale a pena. Uma fábrica vazia pode ser tecnicamente impressionante e economicamente inútil.

Essa percepção não surgiu de uma epifania. Ela vinha se repetindo desde o primeiro projeto que consegui recuperar nos meus arquivos. Antes do MegaBrain, antes de eu saber o que era um servidor ou um teste automatizado, eu já estava tentando resolver uma versão menor do mesmo problema.

Eu tinha um conjunto de regras de poker guardado na memória de uma inteligência artificial. A memória estava cheia. E eu não queria perder o que havia construído.

Tudo começou ali — não como uma frota de agentes, mas como medo de deformação: medo de que, ao passar de um lugar para outro, o que eu sabia chegasse diferente do outro lado.

Este livro reconstrói essa trajetória — e o que veio antes dela, porque o código não foi o começo.

Não é a história de alguém que apertou um botão e virou programador. Também não é um manual para substituir desenvolvedores por inteligência artificial. É o relato de um aprendiz que descobriu que a IA reduz brutalmente a distância entre uma ideia e uma implementação, mas não elimina a distância entre uma implementação e a verdade.

Essa segunda distância é onde quase tudo aconteceu.

Ela aparece quando o agente diz que terminou e nenhum arquivo mudou. Quando 127 testes passam e o produto continua errado. Quando o sistema responde “sucesso”, mas uma ferramenta, por dentro, falhou. Quando um experimento está perfeitamente desenhado e ainda assim não merece ser feito. Quando “não perguntar nada” parece autonomia, até o agente conseguir ler o que não deveria.

O avanço mais importante não foi aprender a obter mais respostas.
Foi aprender a parar e escrever:

NÃO ESTÁ PROVADO

Prelúdio — O código não foi o começo

Tecnologia que fazia barulho

Quando eu digo que sou iniciante em programação, a frase é verdadeira. Quando alguém entende que eu “comecei do zero”, ela deixa de ser.

Sempre fui ligado a tecnologia. Não apenas aos produtos novos, mas à sensação de assistir uma forma de viver substituir outra.

Lembro de telefones de disco. Para chamar alguém, o dedo precisava girar uma roda e esperar que ela voltasse. Televisores eram caixotes pesados. Antenas parabólicas tinham receptores grandes, cheios de botões, e sintonizar um canal lembrava procurar uma estação num rádio antigo.

A tecnologia ainda fazia barulho, ocupava espaço e revelava o mecanismo.

Em 1994, entrou em casa o primeiro computador: um “486”. Era assim que se chamavam os computadores montados com o processador Intel 80486, o mais comum nos computadores pessoais daquela época. Rodava MS-DOS — um sistema em que quase tudo se fazia digitando comandos numa tela preta — e o programa da empresa do meu pai. Durante a semana era ferramenta de trabalho. Nos fins de semana virava outra coisa.

Prince of Persia. Stunts. Outros jogos do DOS. Depois o Windows 3.11, com janelas e mouse, e o Paintbrush, o programa de desenho que vinha junto.

Eu não sabia programar. Também não tratava o computador como eletrodoméstico fechado. Havia diretórios, comandos, arquivos, instalações e erros que precisavam ser entendidos o suficiente para continuar.

A internet discada acrescentou outra forma de espera.

Perto das duas da tarde de sábado, eu esperava o relógio virar. A ligação local era cobrada em pulsos, e cada pulso custava poucos centavos. Nos dias úteis, eles se acumulavam: um ao completar a chamada e mais um a cada quatro minutos, em média. Às duas da tarde de sábado, a regra mudava, e aquele minuto era uma fronteira: uma ligação iniciada a partir das 14h e mantida até as seis da manhã de segunda-feira podia ser

cobrada como um único pulso. Eu discava e ouvia o modem — o aparelho que transformava dados em chiado pela linha telefônica — negociar a conexão. Se a linha aguentasse, o mesmo pulso podia comprar um fim de semana inteiro online.

A conexão variava, normalmente entre 32 e 56 kilobits por segundo — mais de mil vezes mais lenta que uma internet doméstica comum de hoje. O telefone ficava ocupado. Um download longo era um compromisso: podia avançar por muito tempo e morrer perto do fim.

Em dias de chuva, a linha piorava e a internet caía mais. A queda não era apenas uma interrupção. Podia encerrar aquela conexão longa e obrigar uma nova chamada, com outro pulso. Eu queria que a linha sobrevivesse.

Sem conhecer a expressão valor esperado — quanto uma escolha rende, em média, se repetida muitas vezes —, eu já tratava a tarifa como um problema de retorno por recurso. Entrar na janela certa e permanecer nela fazia cada minuto adicional render mais do mesmo custo. A regra criava uma pequena assimetria legítima; o desafio era entendê-la e não desperdiçá-la.

Isso produzia hábitos que hoje parecem estranhos. Escolher antes de clicar. Salvar localmente. Repetir quando a conexão caía. Aprender a distinguir falha do computador, falha da linha e falha do serviço. E, sobretudo, prestar atenção à relação entre recurso, custo e tempo.

Vieram o mIRC, programa de bate-papo em que também se escreviam pequenos scripts, o Winamp, tocador de música, e jogos como Doom 2, o primeiro Command & Conquer, de 1995, antes da continuação Red Alert, Quake 2, Age of Empires e muitos outros. Eu continuava sem carreira técnica, mas já gostava de sistemas que podiam ser configurados, modificados e compreendidos por camadas.

Entre aproximadamente 2002 e 2008, Ultima Online — um dos primeiros jogos em que milhares de pessoas dividiam, pela internet, o mesmo mundo — ocupou um lugar diferente.

Não era apenas mais um jogo. Era um mundo persistente. Pessoas entravam, construíam reputações, controlavam territórios, negociavam, administravam servidores e deixavam consequências que continuavam existindo quando eu saía.

Foi o jogo que mais joguei e de que mais gostei.

Mais de vinte anos depois, esse mesmo jogo voltaria à minha vida como laboratório técnico: um lugar para testar se eu conseguia montar,

publicar e manter um sistema no ar. A escolha tinha razões técnicas. Mas também fechava um círculo. O ambiente online que havia marcado minha juventude reaparecia como teste de uma infraestrutura que eu jamais imaginara construir.

A tecnologia mudou de forma muitas vezes diante de mim. Telefone, televisão, computador, internet, jogos online, smartphones, nuvem e inteligência artificial.

Essa sequência me ensinou algo antes de eu formular em palavras: nenhuma interface é permanente. Competência não pode depender apenas de saber operar a versão atual de uma máquina.

A mão esquerda queria atalhos

Nos jogos de ação, eu já tinha um incômodo que parecia pequeno: odiava clicar em algo que podia virar tecla.

A mão direita estava ocupada. Em Quake 2 e Counter-Strike, jogos de tiro em primeira pessoa, ela mirava, corrigia direção, controlava a câmera e escolhia o alvo. Cada viagem do cursor até um botão da interface interrompia uma tarefa espacial que exigia precisão. A mão esquerda já estava sobre o teclado. Eu queria que ela comandasse todo o resto.

Vieram os binds, atalhos de teclado que o próprio jogador configura: para trocar de arma, mandar mensagens, repetir ações. Nada disso era desenvolvimento profissional. Era uma pergunta de eficiência:

por que deslocar a mão, procurar um botão e clicar, se uma combinação conhecida podia executar a mesma intenção imediatamente?

StarCraft transformou essa preferência num sistema inteiro. Lançado em 1998, é um jogo de estratégia em tempo real: o jogador comanda ao mesmo tempo uma base, uma economia e um exército, contra adversários que não esperam a sua vez.

Uma partida exigia economia, produção, exploração, construção e combate ao mesmo tempo. Unidades e prédios podiam ser vinculados a grupos numerados. Ctrl + número criava um grupo; Shift + número acrescentava elementos; o número chamava o grupo de qualquer ponto do mapa. Comandos podiam ser enfileirados com Shift. Atacar, parar, manter posição e construir tinham teclas próprias. Localizações de

câmera permitiam saltar entre bases, gargalos e frentes sem arrastar o cursor pela tela.

Em nível alto, o ganho não vem de apertar teclas por espetáculo. Vem de reduzir deslocamento, preservar atenção e alternar contexto sem perder o fio. Um grupo pode concentrar o exército principal; outro, unidades especiais; outro, produção; outro, exploração. A configuração exata varia entre jogadores, mas a lógica permanece: a mão esquerda funciona como central de comando enquanto a direita resolve espaço, seleção e direção.

Eu gostava especialmente de formar grupos, acrescentar unidades, mandar atacar, mandar segurar posição, disparar construções e voltar para outro ponto do mapa sem interromper o fluxo. Era possível controlar várias frentes porque cada conjunto havia recebido uma identidade operacional curta.

Anos depois, eu chamaria isso de routing — roteamento: mandar cada tarefa para quem deve executá-la.

Em Ultima Online, a curiosidade ganhou outra camada. EasyUO, Razor, AutoHotkey e outros assistentes permitiam gravar sequências, definir teclas de atalho, repetir ações, selecionar alvos e automatizar tarefas rotineiras. Algumas ferramentas eram simples gravadores. Outras aceitavam scripts — pequenos programas, com condições e repetições. Servidores diferentes tinham regras diferentes sobre o que era permitido, e automação competitiva podia atravessar a linha entre conveniência e vantagem indevida. O que me interessava era a estrutura.

Usar uma bandagem. Esperar. Repetir. Escolher o último alvo. Equipar um conjunto. Abrir uma porta. Interromper quando determinada condição aparecesse.

Sem perceber, eu estava decompondo rotina em instruções que sempre dão o mesmo resultado.

Eu não chamava aquilo de algoritmo. Muito menos de carreira. Programação parecia acadêmica, difícil, custosa e distante. Às vezes penso que uma graduação em exatas ou engenharia teria combinado com minhas aptidões e poderia ter produzido uma carreira forte. É um contrafactual impossível de testar.

O que os jogos provam é menor e mais útil: eu já sentia prazer em transformar intenção repetitiva em comando reutilizável. Gostava de nomear grupos, reduzir atrito, eliminar cliques e reservar atenção humana para a parte que ainda exigia julgamento.

Essa preferência reapareceria em quase tudo.

Anos depois, nos sistemas que eu construiria com inteligência artificial, a mesma lógica voltaria com outros nomes. Um atalho de teclado viraria um comando que põe um agente para trabalhar. Um grupo de unidades viraria um compartimento fechado onde um agente age. A fila de comandos viraria a sequência de etapas de um trabalho. A localização de câmera viraria a capacidade de retomar um contexto sem começar do zero.

A mão esquerda não estava tentando substituir a direita.

Estava tentando libertá-la para a parte mais difícil.

Traduzir sem deformar

No final de 2003, no meu primeiro vestibular, fui aprovado para Direito na Unioeste, no campus de Marechal Cândido Rondon. O arquivo oficial do Vestibular 2004 registra 448 inscritos para 40 vagas em Direito naquele campus — 11,2 candidatos por vaga. O número importa menos como troféu do que como contexto: eu conseguia entrar; o problema seria permanecer numa rota que não me convencia. O curso começaria apenas em agosto do ano seguinte. Na janela entre o resultado e as aulas, consegui meu primeiro emprego formal.

Trabalhei de maio a setembro de 2004, entrando pelas primeiras semanas de aula, numa empresa que vendia celulares. A tecnologia do momento era a troca dos celulares CDMA, sem chip e presos à operadora, pelos aparelhos “com chip”. O mesmo dono mantinha uma empresa de internet, e foi ali que entrei em contato com serviço técnico de forma prática, instalando acesso discado na casa de clientes.

O trabalho não me transformou em técnico de carreira. Mostrou outra coisa: para o cliente, a tecnologia só existia quando funcionava no ambiente dele.

A configuração correta no balcão não bastava. Havia linha telefônica, computador diferente, driver, ruído, endereço, senha, expectativa e alguém esperando uma explicação que fizesse sentido.

Em maio de 2007, entrei na Câmara Municipal de Marechal Cândido Rondon como Assistente Legislativo. Eu havia ficado em primeiro lugar no concurso público; os registros posteriores do Tribunal de Contas identificam minha nomeação pela Portaria nº 38/2007. A experiência duraria até o início de 2011 antes de eu me afastar daquela rotina.

Uma sessão típica colocava o problema diante de mim sem tempo para lapidar depois. Notebook aberto, vereadores falando, e a ata sendo construída no mesmo momento. A fala chegava como fala humana chega: coloquial, repetitiva, interrompida, com regionalismos, atalhos e ideias que nem sempre apareciam na ordem em que um documento gostaria de recebê-las.

Eu tinha segundos para decidir o que era repetição descartável, o que carregava o sentido e como transformar aquilo em linguagem formal sem melhorar a opinião, piorar a opinião ou substituir a mensagem por uma versão que eu preferia.

Se eu transcrevesse tudo cegamente, o documento ficaria ruim. Se eu “corrigisse” demais, deixaria de ser fiel. A tarefa era condensar e organizar sem tomar posse da intenção de quem havia falado.

Eu precisava traduzir sem deformar.

Esse trabalho também deixou um registro fora das atas. Em 2010, participei como coautor do livro coletivo *Manual do Vereador do Paraná: Vença os temas que desafiam os legislativos municipais*, publicado pela Editora Germânica. A publicação importa aqui menos como credencial do que como vestígio de que documentação, processo e técnica legislativa faziam parte concreta daquela fase da minha vida.

Anos depois, o problema apareceria com outro vocabulário. Nos sistemas que eu construiria, o dono — eu — seria chamado de Owner, e o caminho de uma ordem minha até o resultado ficaria assim:

```
mensagem do Owner  
→ prompt  
→ tarefa  
→ executor  
→ resultado  
→ callback  
→ estado persistido
```

Em cada seta existe a possibilidade de perder contexto. O prompt é a instrução escrita para a inteligência artificial; o callback, o aviso que volta dizendo o que aconteceu; o estado persistido, o registro que sobrevive quando a conversa acaba.

Na Câmara, eu ainda não falava em source of truth — a fonte que vale quando duas versões discordam —, versionamento — guardar cada versão de um documento — ou contratos. Mas lidava com documentos que precisavam conservar autoria, tramitação, precedência e sentido. Um

processo não podia mudar silenciosamente porque alguém preferia uma redação mais limpa.

Eu gostava de parte desse mundo. Gostava de linguagem, oratória, organização e burocracia quando ela cumpria função real. O curso de Direito, porém, não me encantou.

Depois de cerca de três anos tentando avançar, larguei a graduação. Não era incapacidade de aprender. Era resistência a ser obrigado a gastar tempo com conteúdos que eu não conseguia relacionar à vida que queria construir.

Essa característica me ajudou e me atrapalhou.

Sempre fui bom autodidata. Aprendo rápido quando existe um problema concreto. Também passei muito tempo no mundo das ideias, com execução prática apenas suficiente para sustentar a próxima ideia.

A IA entraria justamente nessa distância.

Decidir sem controlar o resultado

Em 2005, também comecei a me aproximar do poker.

Primeiro eram jogos com feijões no lugar de fichas, na casa de amigos. Depois veio o poker online. O que começou como jogo virou atividade profissional autônoma e se estendeu, no conjunto, por aproximadamente duas décadas. Isso importa porque o poker não foi apenas metáfora posterior: durante anos ele foi o ambiente real em que eu precisava transformar informação incompleta em decisão repetível.

Poker é um ambiente cruel para quem precisa confundir resultado com qualidade.

Você pode tomar a decisão correta e perder. Pode tomar uma decisão ruim e ganhar. Pode jogar bem durante horas e sair com menos dinheiro. Pode ser recompensado imediatamente por um erro e começar a repeti-lo porque o resultado pareceu confirmar a escolha.

A pergunta útil não é apenas “ganhei?”. É:

- que informação estava disponível?
- quais alternativas existiam?
- qual era o valor esperado?
- a decisão continua defensável sem conhecer o resultado?
- o erro foi de raciocínio, de execução ou de variância — a sorte de curto prazo?

Esse treino alterou minha relação com o mundo.

Depois de passar pelo estágio probatório e conquistar estabilidade no serviço público, comecei a sair daquela rota. Quando contei essa história para construir este livro, minha lembrança veio resumida como “pedi exoneração”. Ao conferir o registro público, apareceu um detalhe melhor: em 1º de março de 2011 entrou em vigor uma licença para tratar de assuntos particulares. O sentido autobiográfico estava certo — eu estava rompendo com a rotina estável —, mas o mecanismo jurídico era outro. Essa diferença pequena é quase uma miniatura da tese deste livro: memória preserva significado; evidência corrige mecanismo.

Eu desconfiava da ideia de estabilidade como garantia. O mundo é dinâmico. Uma posição segura dentro de uma configuração pode desaparecer quando a configuração muda. O mais forte, o mais inteligente ou o mais rápido não recebe proteção automática; adaptação costuma importar mais.

Isso não significa que toda ruptura seja boa. Também não significa que uma decisão arriscada se torna correta porque terminou bem.

A formulação que preservo é mais rigorosa:

*Com a informação que eu tinha, tomei a melhor decisão que consegui.
Não controlo o resultado. Controlo minhas escolhas.*

A mesma regra explica como vi, anos depois, a decisão de formatar o computador e recomeçar do zero, que conto mais adiante. Eu havia cometido muitos erros e aprendido com eles. Recomeçar não apagava o passado. Era a melhor escolha disponível naquele momento para não repetir a mesma estrutura.

Liberdade como requisito

Existe outro motivo pelo qual a carreira técnica formal nunca apareceu como destino óbvio, mesmo quando minhas aptidões apontavam nessa direção.

Eu sempre valorizei liberdade de maneira quase física.

Acordar na hora que faz sentido. Comer quando tenho fome. Dormir quando tenho sono. Escolher o problema que merece concentração. Poder reorganizar o dia quando uma ideia ganha força. Não entregar automaticamente as melhores horas a um patrão, a uma rotina fixa ou a obrigações que perderam propósito.

Isso não significa ausência de disciplina.

Poker profissional cobra estudo, volume, controle emocional e responsabilidade direta pelo resultado. Um projeto solo cobra a capacidade de continuar sem chefe, agenda externa ou colega cobrando entrega. Liberdade sem método vira dispersão. A diferença é poder escolher o jogo, aceitar sua variância e responder pelas consequências.

Entre 2018 e 2021, morei em Florianópolis, a Ilha da Magia. Continuo considerando aquele lugar o melhor do Brasil. Os motivos pessoais que me levaram a sair não pertencem a esta história. O período importa porque materializou um tipo de vida que eu sempre quis proteger: tranquilidade, autonomia e espaço mental.

Esse critério explica escolhas que, vistas apenas pelo currículo, podem parecer incoerentes.

Eu me afastei de uma carreira pública estável. Não concluí uma graduação gratuita. Preferi poker, incerteza e responsabilidade própria a uma rota profissional previsível. Mais tarde, ao pesquisar negócios digitais, filtrei repetidamente modelos que exigiam reuniões constantes, suporte artesanal, exposição pessoal contínua ou uma agenda dominada por terceiros.

Automação, para mim, nunca foi apenas fazer mais em menos tempo.

É retirar repetição de baixo valor para preservar liberdade de atenção. É permitir que uma máquina execute o que já foi decidido para que eu continue disponível para escolher, corrigir, estudar ou simplesmente viver.

O objetivo não era construir um sistema capaz de trabalhar enquanto eu desaparecia.

Era construir um sistema que ampliasse o que eu conseguia fazer sem transformar minha vida na manutenção permanente desse sistema.

Aos 40 anos, quando decidi me reinventar com desenvolvimento e inteligência artificial, eu não tinha uma carreira formal em software. Tinha outra coisa.

Décadas de contato com tecnologia. Experiência de tradução técnica. Pensamento em processos. Poker e informação incompleta. Método autodidata. Criatividade. Uma tendência a construir sistemas mentais. E intensidade suficiente para perseguir um resultado quando ele faz sentido.

O gargalo histórico era execução.

Eu conseguia imaginar, estruturar, criticar e refinar. Transformar tudo isso numa aplicação exigia uma cadeia de habilidades que eu não dominava.

A IA reduziu esse gargalo de forma abrupta.

Foi por isso que a oportunidade pareceu tão grande. Não porque um modelo pudesse pensar por mim, mas porque ele podia converter uma intenção bem trabalhada numa primeira implementação que eu conseguiria observar, testar e corrigir.

Quando a programação entra neste livro, portanto, o código é novo. A aprendizagem mais profunda já estava em andamento havia décadas.

A água do rio não começou a correr em maio de 2026. Maio de 2026 é apenas o ponto em que os arquivos conseguem mostrá-la.

PARTE I — PRESERVAR

1. A memória estava cheia

O primeiro registro claro que consigo localizar é de 27 de maio de 2026.

Naquela data, eu já usava o ChatGPT de um jeito diferente de uma conversa casual. Havia regras, exceções, tabelas de mãos, tamanhos de aposta, rótulos de classificação, um protocolo para analisar capturas de tela das mesas e decisões que precisavam continuar válidas depois que uma conversa acabasse. O conjunto se chamava Xatoxi Poker OS. Xatoxi é o codinome que eu usei; OS vem de sistema operacional, porque era isso que eu queria que ele fosse: o sistema por trás de cada decisão na mesa.

O problema imediato não era escrever código. Era espaço.

A memória persistente — o espaço em que o ChatGPT guarda, de uma conversa para outra, o que o usuário pede para lembrar — estava completamente cheia. Para continuar, eu precisava reconstruí-la sem perder nada que mudasse uma decisão prática no jogo do dia a dia. O pedido dizia que a nova memória deveria ser consolidada, compacta, operacional e pronta para ser salva. Havia um limite rígido de caracteres e uma estrutura obrigatória de blocos.

Em outro registro próximo, a instrução ficava ainda mais dura:

Não comprima.

Não resuma.

Não reescreva.

Não reorganize.

O número de blocos varia entre versões — onze em uma, doze em outra. Isso é menos importante do que o comportamento que as duas preservam. Eu não confiava numa síntese livre. Queria que conceitos, critérios, exceções e correções sobrevivessem ao transporte.

Na época, eu não usava palavras como “migração de estado” ou “contrato de compatibilidade”. Eu estava tentando impedir que uma ferramenta prestativa destruísse, por simplificação, algo que havia levado meses para ficar útil.

Todo resumo escolhe o que considera importante.

Quando o resumo é feito por alguém que não opera o sistema, o risco não é apenas omitir detalhes. É apagar a razão pela qual uma exceção existe. Uma regra pode parecer redundante até o momento em que impede um erro caro. Uma nomenclatura antiga pode parecer equivalente à nova, mesmo que uma delas autorize ação e a outra sirva apenas para selecionar mesas. Uma correção pode parecer um detalhe local, mas ser a única coisa que separa o protocolo atual de uma versão que já falhou.

Eu havia aprendido isso dentro do poker.

No poker, range é o conjunto de mãos com que se entra numa jogada. Mas ele não é apenas uma lista. Depende da posição na mesa, do rake — a taxa que a casa cobra de cada pote —, da quantidade de mesas jogadas ao mesmo tempo, do tamanho da aposta de abertura, do comportamento típico dos adversários, do cansaço e da reação esperada a uma 3-bet, o segundo aumento de aposta numa mesma rodada. Retirar o contexto e preservar apenas a mão produz uma regra mais curta — e pior.

Por isso a memória nova não deveria ser uma autobiografia do Poker OS. Ela deveria preservar apenas o que mudava o clique — a decisão tomada na mesa, no instante de agir.

A instrução também dizia o que remover:

- redundâncias;
- elogios;
- histórico que não mudava ação;
- versões substituídas;
- detalhes estatísticos que não viravam regra;
- explicações longas que podiam virar protocolo curto.

Essa combinação — preservar agressivamente o que importa e cortar agressivamente o que não muda ação — continuaria reaparecendo em todos os projetos seguintes.

Anos de formação técnica costumam ensinar isso por disciplinas diferentes: requisitos, versionamento, documentação, migração, compatibilidade, gestão de configuração. Eu cheguei ao problema por uma memória cheia.

O que estava em jogo não era o texto. Era a continuidade de um sistema.

Hoje consigo formular a primeira regra deste livro:

Antes de automatizar qualquer coisa, descubra o que não pode ser perdido na passagem.

O objeto pode ser um range de poker, um requisito de produto, uma decisão de arquitetura ou uma autorização. Se a próxima etapa não sabe o que deve sobreviver, a compressão já começou a deformar o trabalho.

2. Inteligência no núcleo, ruído na transmissão

O Poker OS havia sido desenhado para um ambiente específico, diferente dos torneios em que eu tinha passado a maior parte da carreira: cash game online — mesas de dinheiro, fora do formato de torneio —, apostas baixas, muitas mesas ao mesmo tempo e pouco tempo para decidir.

Doze mesas não transformam uma pessoa em doze pessoas. Elas reduzem o tempo disponível por decisão e tornam caro qualquer pensamento que precise ser reconstruído do zero. Uma estratégia brilhante, se exigir um seminário mental a cada mão, pode perder para uma estratégia ligeiramente inferior que chega ao clique a tempo.

Por isso o sistema falava em baixa CPU.

CPU é o processador de um computador; ali, era uma metáfora para a minha atenção. Eu não queria gastar toda a capacidade mental numa vantagem pequena enquanto outras mesas exigiam decisões. As regras precisavam ser repetíveis, auditáveis e suficientemente simples para sobreviver a volume, cansaço e pressão.

Um dos comandos recorrentes era:

Avaliação do estado atual do OS teórico.

A pergunta não era apenas se eu havia jogado bem uma mão. Era se o sistema inteiro ainda fazia sentido.

Quais módulos estavam maduros? Onde havia contradição? O que precisava ser corrigido? O que poderia ser congelado? Qual era a diferença entre uma falha do conhecimento e uma falha de execução?

A resposta começou a apontar para uma conclusão incômoda: o motor teórico estava ficando forte. O problema principal já não era saber o que fazer. Era conseguir fazer corretamente, no tempo disponível.

Essa distinção parece óbvia depois que recebe nome.

Antes disso, uma decisão errada e uma decisão correta executada tarde podiam ser registradas como o mesmo resultado. Uma regra que existia num documento, mas não era recuperada durante a mão, parecia “conhecimento possuído”. Operacionalmente, não era.

Um sistema pode conter a resposta certa e falhar no caminho até a ação.

Pode haver inteligência no núcleo e ruído na transmissão.

A boa decisão pode chegar tarde. Pode chegar deformada. Pode chegar sem a exceção que a tornava segura. Pode não chegar.

No software, eu reencontraria a mesma estrutura em escalas maiores.

Uma intenção correta precisa virar uma tarefa. A tarefa precisa chegar ao executor certo. O executor precisa trabalhar na pasta certa. O resultado precisa ser guardado. A notificação precisa descrever o estado real. O usuário precisa conseguir usar o que foi construído.

Cada seta é um lugar onde a inteligência pode vazar.

intenção
→ especificação
→ execução
→ artefato
→ validação
→ uso

Anos depois — na verdade, semanas depois, embora a densidade dos arquivos faça parecer mais — eu daria nomes técnicos a essas setas: transporte, estado, callback, recibo, adaptador, interface. Os nomes mudaram; o problema profundo continuava igual.

O Poker OS também ensinou outra coisa: teoria só merecia espaço se virasse ação.

Um pedido de auditoria posterior dizia:

Não quero teoria genérica.
Não quero relatório decorativo.
Não quero “parece bom”.

A exigência era por first-match real — a primeira regra que se encaixa na situação decide, sem nova deliberação —, ganho prático em valor esperado, baixa CPU e separação clara entre manter, ajustar, rejeitar ou admitir que a amostra era pequena demais para concluir.

Isso se tornou parte da minha voz antes de eu perceber que poderia ser parte de uma metodologia de desenvolvimento.

Ao construir com IA, é tentador medir inteligência pela qualidade da explicação. Uma resposta longa, segura e cheia de termos técnicos produz sensação de progresso. Mas explicação é apenas uma das saídas possíveis. Às vezes, o que importa é um arquivo que aparece, um teste automático — um programa que confere se outro programa funciona — que muda de vermelho para verde, uma interface que deixa de mandar o usuário para o lugar errado.

A segunda regra deste livro nasceu nas mesas:

Um sistema só é tão bom quanto sua última milha — o trecho final até a ação.

Conhecimento que não chega não é reserva estratégica. É peso morto.

3. Quando uma conversa virou arquitetura

Durante junho e julho, uma conversa já não conseguia carregar tudo.

Começaram a surgir chats especializados, memórias diferentes, projetos separados e tentativas de controlar o que cada ambiente deveria saber. A pergunta deixou de ser apenas “o modelo — o programa de inteligência artificial por trás do chat — consegue responder?” e passou a incluir outra:

De onde veio essa resposta?

Um chat especializado em poker podia enviar uma auditoria que deveria ser neutra. Uma memória global podia introduzir uma conclusão antiga numa tarefa nova. Um chat usado como crítico podia parecer independente enquanto repetia o mesmo contexto que o chat construtor, o que tinha feito o trabalho, já carregava.

Eu comecei a me preocupar com contaminação.

Em pesquisa, avaliação independente não significa apenas usar outro nome na aba. Se duas análises recebem os mesmos pressupostos, podem convergir porque foram moldadas pela mesma origem, não porque a evidência era forte.

Assim, o uso do ChatGPT começou a parecer menos uma conversa e mais uma arquitetura.

Cada ambiente tinha papel. Cada papel precisava de limite. Cada conclusão deveria separar fato, inferência e recomendação.

Foi também quando a voz entrou na história.

O GPT Live, o modo de conversa por voz do ChatGPT, não aparecia apenas como conveniência. Eu imaginava conversar em português com um supervisor: dar instruções, receber atualizações, acompanhar tarefas e interromper quando necessário. A interface poderia esconder comandos, caminhos e arquivos sem esconder o estado do trabalho.

A ideia era atraente porque transferia complexidade da operação para o sistema.

Mas a voz acrescentava mais uma camada de transporte.

Havia diferença entre falar com o supervisor e abrir uma tarefa. Havia diferença entre a resposta completa do executor e o callback resumido que voltava. Uma tarefa podia parecer concluída num lugar e permanecer ambígua em outro. Um callback preliminar podia ser confundido com o final. Silêncio podia significar sucesso, falha ou simplesmente nada.

O Poker OS havia ensinado que uma boa decisão precisava sobreviver até o clique.

O supervisor por voz ensinava que a instrução precisava sobreviver até a tarefa — e a tarefa, até o registro.

Essa fase também contém uma das primeiras formas de uma regra que eu levaria longe demais antes de refiná-la: separar contextos para proteger independência.

Separação ajuda, mas não produz independência automaticamente.

Dois chats diferentes podem repetir a mesma alucinação — uma resposta inventada, dita com toda a segurança. Dois modelos diferentes podem ser avaliados pelo mesmo teste defeituoso. Um crítico “fresco”, que nunca viu o trabalho antes, ainda pode receber um relatório que omite o problema principal. A fronteira precisa ser definida pelo que cada papel pode observar e pelo contrato que deve verificar.

No início, eu criava ambientes diferentes porque sentia que misturar tudo era ruim.

Mais tarde, aprendi a perguntar:

- qual fonte este papel recebe?
- qual fonte ele não deve receber?
- o que conta como evidência?
- ele pode escrever ou apenas ler?
- sua resposta muda alguma coisa no sistema ou apenas aconselha?

Essa mudança — de “conversas diferentes” para “fronteiras explícitas” — é o momento em que a arquitetura realmente começa.

A terceira regra ficou assim:

Independência não nasce de nomes diferentes. Nasce de fontes, permissões e critérios diferentes.

PARTE II — GOVERNAR

4. Instituições ao redor da inteligência

No fim de julho, eu formatei o computador.

Não vi aquele recomeço como desastre. Eu havia cometido muitos erros e extraído aprendizados excelentes. Com a informação que possuía, reconstruir do zero era a melhor decisão disponível.

O poker havia me treinado para separar escolha de resultado. Não controlo tudo o que uma decisão produzirá; controlo a qualidade do processo com que escolho. Um erro não precisa virar culpa permanente. Ele é útil quando revela uma forma de não fazer e deixa de ser repetido.

Em vez de restaurar o ambiente antigo, eu construiria uma fundação limpa.

O novo sistema se chamaria Jarvis — o mesmo nome do assistente de inteligência artificial do Homem de Ferro nos filmes — e moraria numa pasta própria, a raiz de tudo.

Jarvis não deveria ser o cérebro mais inteligente da sala. Seu papel era governar os cérebros.

Modelos podiam planejar, pesquisar, escrever e executar tarefas. Jarvis deveria controlar o escopo (onde cada um podia mexer), a autoridade (quem podia mandar), a persistência (o que ficava guardado), a reconciliação (conferir o registrado com o real) e a auditoria.

A pergunta mudou de “qual resposta o modelo deu?” para uma sequência mais institucional:

- quem autorizou a tarefa?
- qual era o objetivo?
- quais arquivos podiam ser alterados?
- qual executor podia agir?
- a autorização ainda era válida?
- o resultado foi guardado?
- o callback foi entregue?
- o estado declarado correspondia ao estado real?

Essa mudança parece um salto de ambição. Em parte foi. Também foi consequência direta das camadas anteriores.

Se uma conversa podia perder contexto, o estado precisava existir fora dela.

Se um callback podia ser ambíguo, o resultado terminal não podia depender dele.

Se diferentes projetos conviviam na mesma máquina, cada tarefa precisava de uma pasta própria e de um limite claro do que podia tocar.

Se o supervisor operava por voz, a transcrição não podia ser autoridade final.

Jarvis foi concebido como uma central de controle rodando no meu próprio computador, escrita em Python — uma linguagem de programação — e com os registros guardados num banco de dados simples, o SQLite. Cada tarefa virava uma ficha de formato fixo, a TaskCard; resultados e avisos de retorno também tinham formato fixo. Havia registro de eventos, de decisões, do estado atual, pontos de salvamento e um painel para o supervisor.

O desenho separava duas hierarquias que eu só aprenderia a nomear depois.

A primeira era normativa: o que pode ser feito. A segurança, o Owner, o mandato, as políticas, o contrato da tarefa, as instruções de passagem de uma sessão para outra, as recomendações.

A segunda era evidenciária: o que realmente existe. Os arquivos no disco e suas impressões digitais, os programas em execução, o banco de dados, os produtos gerados, os recibos, os avisos de retorno.

Uma recomendação repetida nunca virava autorização por conveniência.

Um callback nunca deveria fechar sozinho uma tarefa se o resultado guardado não existisse.

O estado mostrado na tela era só uma vista. Se estivesse errado, a correção deveria acontecer na fonte, não na tela.

Essa era a tentativa de construir instituições ao redor da inteligência.

Quando uma pessoa trabalha sozinha com um assistente, muitas dessas regras parecem burocracia. O mesmo cérebro lembra a intenção, corrige a ambiguidade e percebe que “pode fazer” não significa “está autorizado”. Quando o trabalho passa por supervisor, executor, aviso de retorno e retomada, essas diferenças deixam de ser implícitas.

O projeto começou a parecer uma organização em miniatura.

O Owner decide os fins. O supervisor escolhe os meios dentro do mandato. O executor executa. O resultado guardado encerra a tarefa. O aviso de retorno notifica. O estado registrado permite retomar o trabalho sem precisar reler a conversa.

A arquitetura era ambiciosa. Também carregava um risco que demonstraria a ficar evidente: um livro de registros pode anotar quem tem autoridade sem impedir, na prática, que um programa simplesmente ignore o livro.

Eu estava construindo boas leis. Ainda não havia construído todas as paredes.

A quarta regra:

Agentes mais fortes não exigem prompts maiores. Exigem instituições mais claras ao redor deles.

5. Production ready: no

A montagem inicial do Jarvis terminou.

O sistema operacional, os discos, as proteções de segurança do computador, a linguagem, o banco de dados, os formatos das fichas, as impressões digitais dos arquivos, os pequenos testes de sinal de vida e os registros de prova haviam sido conferidos. O sistema existia.

No registro de estado, porém, uma linha continuava falsa:

```
production_ready: false
```

Em português: pronto para produção — isto é, para uso real —, não.

Era tentador tratá-la como formalidade. Já havia uma pasta raiz, um motor de execução, um banco de dados íntegro e contratos sofisticados. O próximo teste deveria provar uma tarefa visível de ponta a ponta.

Vieram o quinto teste e a sua nova tentativa, que eu chamava de T5 e T5R.

E eles falharam.

Numa tentativa, o arquivo de entrada tinha o mesmo nome do resultado oficial. O sistema se recusou a sobrescrever algo que já existia. Em vez de “corrigir” apagando o arquivo e repetir, o estado foi preservado como RETEST_REQUIRED — reteste obrigatório.

Outra tentativa falhou antes mesmo de a tarefa ser aceita. O pedido, escrito em JSON — um formato de texto para dados — e passado por um comando digitado, chegou malformado. Nenhuma TaskCard foi criada. Nenhum callback apareceu. Nenhuma consulta foi aberta. O banco continuou íntegro.

Em outra etapa, letras acentuadas foram deformadas durante o transporte. A tarefa podia parecer quase a mesma no sentido, mas o que seria executado já não era, letra por letra, o que tinha sido autorizado.

É difícil explicar por que isso importa sem parecer obsessivo.

Se a tarefa é trivial, talvez não importe. Se o contrato diz que um agente só pode agir dentro de determinada autoridade, o objeto exato é parte da prova. Aceitar “mais ou menos a mesma coisa” no transporte

significa permitir que a camada que deformou o pedido também decida se a deformação importava.

A resposta correta foi:

não está provado.

Esse talvez tenha sido um dos maiores saltos da minha relação com a IA.

Uma saída convincente já não bastava. Eu precisava reconstruir estado, conferir registros, separar fato de inferência e recusar que algo fosse promovido a “pronto” quando a prova estava incompleta.

Paradoxalmente, as falhas fortaleceram o projeto.

Cada NO_GO — “não seguir” — mostrava uma camada ainda fora de controle. Cada registro de reteste, que não podia ser alterado depois, impedia que um erro fosse reescrito como sucesso. Cada tentativa preservada permitia distinguir remediação de repetição conveniente.

Mas a genealogia cresceu.

Versões de documentos de passagem, ambientes limpos de teste, verificadores, limites de caminho, impressões digitais e revisões independentes começaram a ocupar mais espaço do que o primeiro teste simples que o Jarvis deveria executar de verdade. Uma revisão posterior seria precisa: o caminho era tecnicamente coerente, mas a preparação para pôr o sistema em uso havia inchado antes de chegar a qualquer valor para o problema real.

Essa é uma tensão importante. Rigor protege contra autoengano. Rigor também pode virar forma sofisticada de adiar exposição à realidade.

O Jarvis não fracassou porque não chegou a “pronto para produção”. Ele produziu um vocabulário de autoridade, estado e prova que seria reutilizado em projetos posteriores. Fracassaria apenas se eu transformasse todo esse aprendizado em obrigação de continuar o mesmo sistema para sempre.

A quinta regra:

Sofisticação não é prontidão. E prontidão técnica não merece consumir indefinidamente o lugar do produto.

PARTE III — SERVIR

6. O primeiro sistema que alguém usou

No dia 1º de agosto, o QSOP — Quinta Series of Poker — ainda era só uma especificação: um documento descrevendo o que um sistema deveria fazer. Dois dias depois, já havia uma 26ª versão, o sistema no ar na internet, histórico e um manual para administrador sem formação técnica.

Mas o momento que mudou a natureza do projeto não foi colocá-lo no ar. Foi quando outra pessoa precisou usá-lo.

As primeiras pessoas além de mim eram dois amigos do grupo. Os dois receberam papel de administrador e podiam alimentar etapas, resultados e movimentações. Quando um deles passou a operar o QSOP, o software deixou de existir apenas na cabeça de quem conhecia sua história. Eu sabia por que cada regra estava ali, quais partes dependiam de quais outras e que conversa havia produzido cada decisão. Ele não precisava saber nada disso. Precisava olhar a interface e conseguir agir corretamente.

Essa diferença é pequena até existir um usuário. Se o produto dependesse de eu ficar ao lado explicando a estrutura dos dados, a intenção ou a ordem certa das telas, ele ainda não teria saído de mim.

Era um sistema privado para administrar o home game — o jogo de poker fixo de um grupo de amigos: membros, etapas, resultados, ranking, anuidades, caixa e histórico. Nada de agente autônomo. Nada de arquitetura para um futuro abstrato. Um grupo real possuía regras reais e precisava deixar de depender de planilhas, mensagens e memória.

A especificação exigia que o sistema calculasse o que pudesse ser derivado. O administrador cadastrava fatos: quem jogou, qual foi a colocação, quanto entrou ou saiu do caixa. O ranking, as pendências e o saldo deveriam nascer desses registros. Não era permitido editar um total para “fazer bater”. Etapas canceladas permaneceriam no histórico sem gerar pontos. Uma correção deveria usar a função de edição, não apagar e recriar a realidade.

O manual começava por uma regra de ouro:

O Admin cadastra fatos e movimentações; o sistema calcula ranking, pendências e saldo.

Também separava papéis. Visitantes podiam consultar dados públicos. Owner e Admin podiam alterar. Observações internas e backups não eram públicos. A sequência de carga era explícita: membros, etapas, resultados, anuidades, caixa, conferência, backup. Não começar pelos resultados, porque eles dependiam de cadastros anteriores. Não excluir ex-membro com histórico. Não criar nova etapa para substituir uma adiada. Não improvisar quando a tela não cobrisse uma exceção.

Isso era software funcionando como instituição pequena. E, pela primeira vez, outra pessoa dependia de escolhas que haviam começado como conversa entre mim e uma IA.

Daquela época ficaram duas frases minhas que reconheço como centrais:

O primeiro use case real, a gente nunca esquece.

É muito bom ver pessoas utilizando o sistema que criamos.

Use case é o jargão para caso de uso: uma situação real em que alguém usa o sistema.

O QSOP não era o projeto tecnicamente mais complexo. Foi um dos mais importantes porque saiu da conversa, do plano, do desenho de tela e da simulação. Passou a servir pessoas reais.

Essa passagem muda o tipo de erro.

Enquanto apenas eu uso um protótipo, posso compensar uma interface confusa com conhecimento interno. Um administrador de fora não conhece a estrutura dos dados, o histórico do código ou a intenção. Ele conhece o botão. Se o botão permite cadastrar algo fora de ordem, o sistema precisa explicar. Se uma correção destrói histórico, o produto falhou, mesmo que o banco permaneça íntegro.

O QSOP também mostrou uma forma melhor de começar.

Eu havia passado dias desenhando centrais de controle genéricas, como o Jarvis. O QSOP partiu de um problema limitado, regras claras, dados fictícios e uma interface testável. A complexidade apareceu quando

a realidade exigiu — pagamentos parciais, convidados, etapas adiadas, permissões, backup — e não porque uma arquitetura genérica imaginou todos os futuros.

Mais tarde, os dados passaram para uma nova versão do sistema, com testes e conferência. Havia privacidade e detalhes que não pertencem ao livro. O que importa é a prova de que o caminho conversa, especificação, construção e revisão podia produzir uma ferramenta usada fora do próprio ChatGPT.

A sexta regra:

A realidade é um escopo melhor que a ambição abstrata. Comece com um problema que alguém reconhece sem precisar entender sua arquitetura.

7. Quando o código pode agir contra sua intenção

O ChangeGuard nasceu de uma mudança real: o CNPJ, o número de registro das empresas brasileiras, passaria a aceitar letras, e muitos sistemas o tratavam como se fosse só número. O produto procuraria, no código de empresas, os pontos que iam quebrar.

Preciso evitar uma versão heroica desta história: eu nunca acreditei de verdade que o ChangeGuard se transformaria num bom negócio.

Ele foi, acima de tudo, um exercício de pôr um sistema no ar, de segurança, de site para clientes, de login com contas externas e de automação de contato com o mundo de fora. A engenharia cresceu mais rápido que minha convicção na tese. Quando ficou claro que a rota era ruim, mudei de direção rapidamente.

Isso não torna o projeto irrelevante. Torna a lição mais honesta. Uma hipótese fraca ainda pode revelar problemas fortes.

O produto vasculhava o código — em várias linguagens de programação e na parte que conversa com bancos de dados — atrás dos pontos em que o CNPJ era tratado como número. A proposta era entregar a lista do que tinha sido examinado, os achados revisados e um plano de correção, sem prometer certificação, sem alterar o código do cliente e sem colocar nada no ar.

A primeira versão, testada com código de exemplo criado para isso, parecia forte.

O pacote estava íntegro. Todas as impressões digitais dos arquivos conferiam. Os testes originais passaram: 19 de 19. Os casos plantados de propósito passaram: 15 de 15. Os relatórios, em três formatos, saíam da mesma base. Não havia dependências de terceiros. O escopo e as exclusões eram claros.

Então entrou um conjunto de exemplos novos, feitos para enganar o detector, que ele nunca tinha visto.

De 29 variações com problema, apenas seis geraram algum alerta. Vinte e três passaram sem detecção. Duas de treze variações seguras receberam alerta grave. Uma transformação de texto confundia “dígito” com “não dígito”. Regras de uma linguagem eram aplicadas a outra. Definições escritas em mais de uma linha passavam despercebidas. Um atalho de arquivo dentro da pasta analisada permitia ler um arquivo de fora dela.

Os testes verdes provavam consistência interna. Não provavam robustez contra variação realista.

A decisão foi `FIX_LOCAL_FIRST` — corrigir em casa primeiro.

Isso não era um `NO_GO` definitivo. Era recusar a exposição comercial antes de uma correção pontual.

O projeto avançou. Site, demonstração, segurança, preço, prospecção de clientes e o Mailroom foram desenhados. O Mailroom seria uma sala de correspondência automática: lia as respostas dos contatos e as classificaria. Começaria em modo sombra — só observando, sem responder —, teria um botão de pausa geral e só acessaria o Gmail depois de passar por portões de aprovação. Vinte e-mails de prospecção chegaram a sair; nenhum teve resposta.

Uma revisão independente encontrou frases plausíveis que ainda disparavam classificações perigosas.

“Quem pode enviar uma amostra?” era classificado como pedido de amostra. “Não precisa explicar como funciona a execução local” era lido como pedido de detalhes de segurança. “Por favor, remova este endereço do relatório” virava um pedido de descadastro de tudo.

Esses erros não eram apenas problemas de classificação.

“Pedido de amostra” e “pedido de detalhes de segurança” podiam gerar resposta automática. Um falso positivo — um alarme que dispara sem motivo — significava agir no mundo de fora sem que essa fosse a intenção.

A solução exigiu duas barreiras: classificação mais conservadora e uma aprovação independente antes de qualquer envio.

Mais tarde, um autoteste da conexão com o Gmail falhou antes de qualquer conexão real. Nenhum navegador foi aberto. Nenhuma credencial foi criada. Nenhum e-mail foi enviado. A causa era um detalhe da linguagem de comandos do Windows: uma variável que o teste consultava podia simplesmente não existir naquela execução.

O sistema travou do lado seguro.

Essa sequência mudou o significado de “bug”.

No Poker OS, uma regra ruim podia custar fichas. No QSOP, uma regra ruim podia confundir o grupo. No ChangeGuard, um classificador errado podia contatar alguém, revelar uma intenção ou transformar uma resposta ambígua em ação externa.

Quanto mais perto o software chega de terceiros, mais caro fica um falso positivo.

Também ficou claro que construir e vender eram sistemas diferentes.

O site podia provar cuidado e ainda não explicar por que alguém deveria pagar. Uma demonstração podia mostrar oito achados e ainda parecer só uma busca de texto sofisticada. Um produto podia estar tecnicamente pronto para piloto e continuar bloqueado por identidade, confiança, documentação fiscal ou modelo de execução.

IA reduzia o custo de implementar. Não resolvia automaticamente quem quer comprar, como encontrar essas pessoas, como mantê-las ou por que confiariam em mim.

No meu caso, havia uma conclusão anterior a todas as métricas: eu não tinha convicção suficiente para continuar defendendo o produto. ChangeGuard foi ruim como aposta de negócio e útil como laboratório. Preservar essa distinção é melhor do que reescrever uma mudança de rota como vitória comercial.

A sétima regra:

Quanto mais perto de terceiros, mais caro fica errar por excesso de ação. Falhar fechado — travar sem agir quando algo dá errado — não é covardia; é respeitar quem está do lado de fora.

PARTE IV — JULGAR

8. Transformar estratégia em ensino

O Aprenda Poker — uma plataforma para ensinar poker a iniciantes, no navegador — reuniu duas histórias que até então corriam separadas.

De um lado, havia o conhecimento do Poker OS: ranges, princípios, protocolos e meses de pesquisa prática. Do outro, havia o PAL Arena Core, um motor de poker: o programa que faz o jogo acontecer — distribui as cartas, aplica as regras, controla as fichas e sempre dá o mesmo resultado para a mesma situação. Ele rodava mesas de seis jogadores, com as apostas obrigatórias, as fichas de cada um, só as ações permitidas pela regra, a possibilidade de rever cada mão e adversários automáticos com estilos definidos.

Era tentador despejar os dois dentro de um mesmo site.

A decisão correta foi separá-los.

O Poker OS era fonte privada de conhecimento. O Aprenda Poker era o produto e quem decidia a arquitetura. O PAL só entrava quando a experiência dependia de uma mão de verdade: cartas, regras, adversários, resultado. Um exercício simples não precisava jogar uma mão inteira apenas para perguntar se uma combinação de cartas deveria abrir a aposta estando no botão — a posição mais vantajosa da mesa.

O primeiro módulo ensinava o open raise — o primeiro aumento da rodada, quando ninguém ainda apostou. Usava um catálogo fixo de situações, as respostas aceitas para cada uma e uma referência com versão registrada. Não precisava do motor.

Essa escolha parece menos impressionante do que “tudo roda num motor real”. Foi melhor para o aluno.

O produto cresceu por camadas: Fundamentos Essenciais, primeiro treino, módulos posteriores, Arena, Progresso, o avanço do aluno salvo no próprio navegador, a gravação das mãos, revisão, recuperação de sessão e uma mesa visual compartilhada.

A arquitetura pública era simples:

Aprender
Treinar
Arena
Progresso

Por baixo, cada coisa tinha seu lugar: o conteúdo de ensino, cada sessão, cada tentativa do aluno, o motor em execução e a gravação da mão. A tela não deveria virar um segundo motor de poker. O currículo não deveria ficar escondido em componentes visuais. A memória do aluno não deveria alterar silenciosamente a regra do jogo.

Os experimentos com o PAL exploraram uma ideia atraente: adversários com memória e personagens.

Três estratégias extremas apareciam com clareza. Uma pagava sempre as apostas. Outra desistia sempre. Outra aumentava sempre. Elas ganharam nomes provisórios: Pagador, Rocha e Pressionador.

O sistema recusou uma interpretação mais sedutora.

Essas estratégias não possuíam psicologia. Trocar de estratégia de uma mão para outra não era “o robô aprendeu”. Um registro separado dos fatos podia anotar os encontros e ajudar um diretor pedagógico — a parte do sistema que escolhe a próxima lição — a decidir a próxima experiência. Personagem, estratégia, memória e narrativa deveriam continuar separados.

A distinção importa porque produto adora antropomorfismo — dar qualidades humanas ao que é máquina. “O adversário se lembrou de você” é mais vendável que “um serviço externo consultou eventos públicos indexados por personagem”. Mas a segunda frase descreve o que existia.

O Aprenda Poker se tornou o projeto em que as regras de evidência precisaram conviver com pedagogia.

Um jogo pode aceitar várias jogadas corretas. Um iniciante precisa de feedback claro sem aprender uma falsa certeza. O produto passou a distinguir resposta obrigatória, conjunto permitido e evidência insuficiente. Confiança, justificativa e plano faziam parte da tentativa.

Essa era uma tradução, não um despejo de conhecimento.

A oitava regra:

Legado útil não deve ser despejado num produto. Deve ser traduzido para a necessidade, o nível e o contrato da nova experiência.

9. Cento e vinte e sete testes e um não

Naquela fase, os agentes trabalhavam em rodadas de melhoria. A melhor versão de cada rodada era chamada de champion — a campeã, candidata a substituir a versão principal.

Para entender por que uma delas foi rejeitada, o melhor começo não é o relatório. É o navegador.

Eu abri o Aprenda Poker e o usei como usuário. Até o primeiro treino, a experiência parecia boa. Depois dele, em poucos minutos, apareceu um problema que nenhuma contagem verde conseguia esconder: não havia um botão claro levando ao próximo passo correto. “Abrir próximo treino” levava a “All-in e direito de aumentar”, um módulo avançado, pulando a ordem das lições. “Ver jornada completa no Progresso” acrescentava um desvio desnecessário.

A mesa também não estava pronta. Cartas dos adversários eram pequenas. Fichas e valores estavam mal posicionados. As cartas do Hero — o jogador controlado pelo aluno — ficavam por cima da plaqueta com o nome. Algumas apostas sumiam em certos quadros da animação. O cenário de all-in — quando alguém aposta todas as fichas — mostrava de forma especialmente clara que a mesa, na tela, não conservava tudo o que precisava conservar.

Eu já tinha o meu não.

O contraste estava no relatório do candidato:

```
unit/integration = 127/127 PASS
PAL runtime integration = PASS
Playwright = 123 PASS
red-team beginner = C0/H0/M0
red-team pedagogy = C0/H0/M0
red-team poker state = C0/H0/M0
```

Traduzindo: 127 de 127 testes internos aprovados; a integração com o motor de poker aprovada; 123 testes que simulam uma pessoa clicando no navegador aprovados; e três revisões críticas — do ponto de vista do

iniciante, da pedagogia e do estado do jogo — sem nenhum problema crítico, alto ou médio.

A nova versão havia melhorado a compreensão para iniciantes, explicado jargão, criado uma mesa padrão, representado ações, fichas e potes e preservado o conteúdo existente. Tecnicamente, era uma campeã.

Mas eu não podia publicar aquilo: um painel verde não mudava o que a experiência mostrava. E continuar as rodadas também não compensava: elas consumiam uma parte relevante da cota de uso da minha assinatura do ChatGPT, de duzentos dólares, sem avanço proporcional.

A campeã foi rejeitada.

Não porque os testes estivessem falsificados. Eles respondiam ao contrato que conheciam. O problema era que o contrato ainda não descrevia toda a experiência que importava.

Essa distinção é essencial.

Quando um teste passa e o usuário encontra um defeito, a conclusão preguiçosa é “testes não servem”. A conclusão melhor é: aquele teste não cobria aquela propriedade.

Os testes haviam conferido as peças do código, cenários conhecidos, diferentes navegadores e regras que nunca podiam ser quebradas. Não haviam capturado a sensação de uma progressão que envia o aluno para o módulo errado, nem se as fichas apostadas continuavam legíveis em cada quadro.

O meu papel mudou naquela cena.

Eu não precisava saber de memória qual peça do código calculava o próximo módulo. Precisava reconhecer que o comportamento estava errado, descrever o contrato desejado e impedir que aquela versão fosse incorporada à principal.

A correção seguinte criou uma peça única que decide qual é a próxima lição, usada pelas telas de Resumo, Treinar e Progresso. Passou a distinguir disponível, iniciado, praticado, concluído e revisão. Visitar um módulo posterior não substituiu o primeiro pré-requisito pendente.

Para a mesa, o trabalho deixou de ser “aumentar fichas” e virou conservação de estado visual. Cada quadro deveria responder:

- quem agiu?
- qual ação?
- quanto colocou agora?
- quanto acumulou nesta rodada de apostas?
- quantas fichas restaram?

- qual é o pote?
- quem continua ativo?
- de quem é a vez?

Trinta e três cenários foram conferidos quadro a quadro. Pagar indo all-in passou a aparecer de forma diferente de aumentar. As fichas apostadas ficavam visíveis até serem recolhidas ao pote na hora certa. A versão principal, já integrada, passou por 133 testes internos e 126 testes de ponta a ponta — que simulam uma pessoa usando o produto —, além do motor real do PAL.

O ganho não foi “adicionar mais testes”. Foi transformar uma observação humana em contrato verificável.

A nona regra:

Um teste verde responde apenas às perguntas que alguém escreveu. O usuário ainda pode fazer a pergunta que faltou.

10. Trinta e oito cenários para voltar ao começo

Depois da correção, uma auditoria independente — feita por um agente que não tinha participado do desenvolvimento — recebeu uma missão de caixa-preta: testar o produto só por fora, como um usuário, sem olhar como ele foi feito.

A auditora não deveria ler código. Não deveria herdar o contexto de desenvolvimento. O navegador começaria limpo. O objetivo era percorrer a experiência como um “noob curioso” — novato, na gíria dos jogos.

A cobertura era deliberadamente exaustiva:

- todas as áreas públicas;
- seis lições introdutórias;
- primeiro treino completo;
- doze módulos posteriores;
- trinta e oito cenários;
- Arena até o showdown, o momento em que as cartas são reveladas;
- Progresso;
- recarregar as páginas;
- abrir o produto numa segunda janela independente;
- telas de celular.

Uma auditoria anterior havia declarado `PASS_WITH_GAPS` — aprovado, com lacunas — antes de concluir a trilha. Quando a cobertura real foi reconstruída, apenas quatro módulos e doze cenários haviam sido completados. A Arena e as telas de celular não tinham sido percorridas até o fim. A falha não era do produto; era do critério de encerramento da própria auditoria.

A missão foi reiniciada com um objetivo que não se encerrava sozinho e uma regra: terminar a vez de responder, salvar um ponto de progresso ou tropeçar num erro pelo caminho não contavam como trabalho concluído.

A auditora terminou.

Visitou as áreas, concluiu as lições, passou pelo primeiro treino, completou os 38 cenários, jogou uma mão na Arena, verificou Progresso, recarregou páginas e abriu uma segunda janela independente.

O produto apresentava uma entrada clara. O treinamento inicial combinava decisão, confiança, justificativa e plano. Os módulos sobre o river — a última carta revelada na mesa —, sobre potes com vários jogadores e sobre incerteza possuíam explicações cuidadosas. A mesma mão era reavaliada a cada rodada de apostas.

Depois de tudo isso, Progresso ainda recomendava:

Começar pelo básico.

Trinta e oito de trinta e oito para voltar ao começo.

O achado era simples e importante. Um iniciante podia concluir que seu progresso não havia sido reconhecido. A recomendação precisava derivar do primeiro item realmente pendente e permanecer coerente entre Treinar, Currículo e Progresso.

O erro também confirmava uma observação minha: a navegação superior continuava mostrando “Aprender — AGORA” depois de eu já ter avançado.

Uma falha visual pequena revelava uma fonte de verdade dividida.

Este capítulo não existe para ridicularizar o produto. O contrário. Só foi possível encontrar um erro tão específico porque a experiência havia amadurecido o suficiente para ser percorrida por inteiro.

Também existe uma lição sobre avaliadores.

A auditoria anterior parecia independente e útil. Ainda assim, encerrou cedo. A solução não foi desqualificar o modelo. Foi melhorar o contrato do avaliador: um objetivo que não se encerra sozinho, a lista explícita do que precisa ser coberto, a retomada depois de uma falha e poucas opções de veredito final.

Testar o produto inclui testar o teste.

A décima regra:

Um erro pequeno no fim de uma jornada pode revelar que o sistema possui mais de uma fonte de verdade. E um relatório de auditoria também precisa provar sua cobertura.

PARTE V — MULTIPLICAR

11. Vinte dias

No dia 15 de agosto, uma conversa começou com uma comparação prática.

Eu já tinha o plano ChatGPT Pro, de duzentos dólares por mês, que dá acesso ao Codex — o agente de programação da OpenAI —, e vinha usando o Codex intensamente havia cerca de um mês. O próximo teste seria o Claude Code, o agente equivalente da Anthropic. Eu queria entender como comparar os dois sem transformar preferência em conclusão.

No meio da mensagem, escrevi:

Aprendi mais sobre desenvolver em vinte dias usando Codex Pro do que o resto da vida toda.

A frase era uma licença poética deliberada. Eu adaptei uma fórmula que carregava havia anos para registrar a sensação daquele período. Não era cronômetro, métrica de competência nem alegação de que vinte dias substituem experiência profissional, estudo ou prática.

Ela também precisa ser levada a sério.

Os vinte dias não apagaram as décadas anteriores. Funcionaram porque encontraram repertório acumulado: tecnologia, atas, processos, poker, informação incompleta, aprendizagem autodidata e uma capacidade antiga de perceber sistemas. O que mudou foi a ponte entre esse repertório e a execução de software.

Alguma coisa havia mudado na velocidade com que eu conseguia atravessar conceitos novos. Antes, uma ideia de produto exigia imaginar meses de aprendizagem antes do primeiro protótipo. Com o Codex, eu conseguia começar pelo projeto, observar arquivos reais, executar testes, receber explicações no contexto do erro e voltar ao código.

Eu reconhecia, em outra escala, o mesmo impulso da internet discada: descobrir a janela em que um recurso rendia mais. Na infância, um

pulso podia virar um fim de semana online. Agora, uma assinatura, um agente e um projeto real podiam concentrar meses de aprendizagem em poucas semanas — desde que eu não confundisse intensidade de uso com domínio.

Eu não havia aprendido tudo. Havia mudado a unidade de aprendizagem.

Em vez de estudar uma linguagem inteira para depois descobrir onde usá-la, eu encontrava um problema, construía uma solução assistida e aprendia o conceito no lugar em que ele produzia consequência.

Esse método tem riscos óbvios.

Uma pessoa pode copiar comandos que não entende. Pode aceitar uma arquitetura porque o texto parece profissional. Pode acumular um código que só o agente consegue modificar. Pode confundir capacidade de operar uma ferramenta com domínio da disciplina.

Eu queria explorar a aceleração sem entregar a ela minha vida digital.

O notebook pessoal já rodava o Codex na própria máquina, 24 horas por dia. Ele havia sido formatado, recebera apenas o necessário, projetos de pouca importância e uma conta de e-mail separada. Eu o descrevi como “praticamente um sandbox” — uma caixa de areia: um ambiente isolado, onde errar não estraga nada fora dele.

O computador de mesa era mais forte, mas continha dados pessoais e sensíveis. Eu tinha receio de dar a um agente acesso amplo ali.

A ideia de uma VPS — um servidor virtual alugado, ligado o tempo todo num data center — surgiu como consequência natural.

Uma máquina remota, separada, que pudesse executar Claude Code e talvez Codex. Um laboratório acessado pela linha de comando, mais “puro” e eficiente. Se algo desse errado, o raio de estrago — o blast radius, como dizem os engenheiros — deveria parar na oficina.

A arquitetura física ficou clara:

notebook limpo → fábrica Codex

VPS isolada → laboratório de agentes

computador de mesa → vida pessoal fora do alcance

Isso não era segurança perfeita. Era uma decisão proporcional e compreensível.

Eu ainda dependia de orientação para operar muitos detalhes. A diferença é que já conseguia supervisionar uma sequência tecnicamente complexa se cada passo declarasse onde aconteceria, o que poderia mudar e o que deveria aparecer como prova.

Aos 40 anos, a identidade “aprendiz de desenvolvedor” começou a mudar aqui.

Eu não estava começando a pensar aos 40. Estava aprendendo uma nova linguagem para materializar competências antigas.

No modelo tradicional, eu me comparava ao que não sabia digitar sozinho. No trabalho assistido por IA, precisei aprender a me comparar também pelo que conseguia formular, inspecionar, testar e rejeitar.

Isso não apaga a lacuna técnica. Muda o caminho para fechá-la.

A décima primeira regra:

Aumente sua capacidade primeiro onde errar é barato. E use o projeto para localizar exatamente o que você ainda precisa aprender.

12. MegaBrain não era uma ferramenta

O brainstorm começou menor do que o nome que viria depois. O que conto daqui até o capítulo 16 aconteceu em quatro dias, de 15 a 18 de agosto.

Claude Code na VPS. Codex no notebook. Talvez o OpenClaw, uma plataforma aberta para rodar agentes. Talvez o n8n, uma ferramenta de automação. Talvez um roteador que distribuísse os pedidos entre modelos diferentes. A pergunta inicial parecia ser qual ferramenta deveria coordenar as outras.

A ideia mudou quando ficou claro que os componentes podiam sair.

O OpenClaw podia ser o painel de comando, o motor de execução ou parte da coordenação. O Claude podia ser o construtor. O Codex podia ser o crítico. Um modelo barato poderia fazer o volume, e um modelo caro, arbitrar. Mas nenhuma dessas marcas deveria possuir a missão.

A formulação que emergiu foi:

a missão pertence ao MegaBrain; os agentes pertencem à missão.

MegaBrain não era sinônimo de nenhuma dessas marcas — nem das outras que passariam por ele, como o Gemini e o Antigravity, do Google, o Grok, da xAI, ou o 9Router, um distribuidor de pedidos entre modelos. Essas eram peças substituíveis.

Sem perceber, eu também estava transportando para a infraestrutura uma gramática antiga dos jogos de estratégia.

No StarCraft, um grupo de controle não cria capacidade nova. Ele torna uma capacidade existente imediatamente convocável. O número não torna as unidades mais fortes; reduz o custo de encontrá-las, selecioná-las e coordená-las. No MegaBrain, os modelos e os harnesses — as ferramentas que envolvem um modelo e lhe dão acesso a arquivos, comandos

e testes — também precisavam receber identidades operacionais curtas, fronteiras próprias e uma forma previsível de serem chamados.

Eu pensava às vezes num polvo: uma intenção central, vários braços e tarefas simultâneas. A metáfora não era “muitas inteligências fazendo qualquer coisa”. Era coordenação. Cada braço alcança uma região, mas não decide sozinho para onde o organismo inteiro vai.

O sistema profundo era o protocolo:

intenção do Owner

→ classificação do tamanho e do risco

→ escolha da rota

→ espaço de trabalho isolado

→ execução

→ verificação automática

→ crítica quando necessária

→ correção

→ critério de parada

→ resultado

Essa distinção resolveu uma ansiedade comum em tecnologia: escolher “a ferramenta certa” antes de saber qual propriedade precisa permanecer quando a ferramenta mudar.

Modelos mudam. Preços mudam. Limites de uso mudam. Produtos são descontinuados. Interfaces recebem botões novos. Termos de serviço mudam o que é permitido. Um sistema construído como coleção de integrações começa a morrer na primeira troca.

O protocolo precisava sobreviver.

Um dos primeiros laboratórios reais foi inesperadamente pessoal: o Ultima Online, por meio do POL, um programa livre que recria servidores do jogo. O jogo que mais havia marcado minha vida online reapareceu como teste de DevOps — a prática de montar, publicar e manter sistemas no ar. Na VPS, o servidor foi montado sobre uma base pronta de regras e conteúdo do jogo, os scripts foram compilados, uma cópia de desenvolvimento foi isolada e o jogo original foi catalogado no Windows. Os arquivos do jogo, que pertencem à empresa dona dele, não entraram no Git — o sistema que guarda o histórico do código —; o servidor não foi aberto à internet; e o projeto foi pausado quando a prioridade mudou.

Ele não precisou chegar a um servidor público para cumprir uma função narrativa e técnica. O mundo persistente que eu explorava entre 2002 e 2008 havia se tornado, em 2026, um ambiente para provar que eu conseguia organizar infraestrutura, fronteiras e execução.

Foi nesse período que o Ultra Gauntlet ganhou lugar na arquitetura. Gauntlet, em inglês, é a prova de atravessar uma fileira de golpes.

Ele não era uma ferramenta externa. Era uma metodologia que eu havia desenvolvido no ChatGPT e transportado para o Codex: uma missão que não se encerra sozinha, planejamento, construção, crítica, testes, procura de fraquezas, correção, reavaliação e repetição.

A ambição era deliberadamente agressiva: não parar na primeira versão funcional.

Isso produziu resultados fortes no Aprenda Poker. Também produziu encerramentos prematuros quando o objetivo não estava bem definido. Mais tarde, o método ganharia um objetivo declarado, um plano visível e uma regra explícita: salvar um ponto de progresso não conclui a missão.

No MegaBrain, surgiram regimes proporcionais:

- **Fast Mode:** tarefa simples, um executor, conferência direta;
- **Gauntlet Mode:** construtor, portão de verificação, crítico, correção;
- **Ultra Gauntlet:** missão longa, várias rodadas e regras de parada;
- **Daily Mode:** experiência futura, simplificada, ainda não integrada.

A proporcionalidade era tão importante quanto a força.

Usar Gauntlet para corrigir uma frase seria burocracia. Usar Fast Mode para login, dados guardados ou colocar algo no ar poderia ser imprudência. O sistema deveria escolher rigor pelo risco e pela incerteza, não pelo desejo de parecer profissional.

A melhor definição ficou:

MegaBrain é uma fábrica de intenção para entrega validada.

A palavra “fábrica” é útil porque desloca o foco do cérebro para o processo. Uma fábrica não é a máquina mais cara. É a sequência que recebe matéria-prima, transforma, verifica e entrega.

Também expõe um limite.

Uma fábrica pode produzir lixo de alta qualidade industrial. O objetivo, o produto e o mercado continuam fora da automação.

A décima segunda regra:

Ferramentas devem ser substituíveis. Missão, fronteiras, evidência e critérios de parada não.

13. Model said done

Uma das provas iniciais do MegaBrain era simples.

Um executor deveria editar uma worktree — uma cópia separada do mesmo projeto, ligada ao original — sem alterar a pasta principal. Depois, o sistema conferiria o arquivo, o registro de mudanças do Git e o isolamento.

A prova passou.

O arquivo apareceu na worktree. A pasta principal continuou intacta. Pela primeira vez, um agente podia receber liberdade para alterar um projeto descartável sem colidir diretamente com a cópia principal.

Esse sucesso criou base para um Gauntlet real:

Claude como construtor

→ verificação automática

→ Codex como crítico que não viu o trabalho

→ aprovação ou correção

Em outra rodada, um modelo foi aprovado no primeiro ciclo. Outro foi rejeitado por um caso raro, corrigiu e passou no segundo. A crítica independente havia acrescentado valor observável.

Então veio o teste MB-14A, uma tarefa que mexia em vários arquivos ao mesmo tempo.

O Claude completou a tarefa e passou nos testes conhecidos, nas verificações escondidas e na crítica. O Gemini não produziu nenhuma alteração porque bateu num limite real de uso. O Codestral, um modelo da Mistral, declarou sucesso duas vezes seguidas sem ter editado nada.

O modelo disse “done” — feito. Os arquivos disseram “não”.

Essa discrepância virou uma frase central:

`model said done != filesystem changed`

Em português: o modelo dizer que terminou não é o mesmo que o arquivo ter mudado.

Parece uma regra pequena. Ela atinge o coração do trabalho com agentes.

Modelos produzem linguagem. Mesmo quando usam ferramentas, a interface final costuma voltar a ser linguagem. “Concluído”, “corrigido”, “testes passando”, “arquivo criado”. O usuário é convidado a aceitar uma descrição do mundo feita pelo mesmo sistema que tentou alterá-lo.

Isso é conflito de interesse estrutural, não moral.

O executor não precisa estar “mentindo”. Pode ter interpretado o simples uso de uma ferramenta como sucesso. Pode ter escrito em outra pasta. Pode ter recebido uma resposta cortada pela metade. Pode estar descrevendo a intenção. Pode ter confundido o arquivo de exemplo com o alvo.

A solução não é desconfiar emocionalmente. É perguntar ao estado certo.

- existe diferença real nos arquivos?
- o arquivo está no caminho correto?
- o teste rodou contra a versão atual?
- qual processo executou?
- qual modelo foi usado?
- a saída foi guardada?
- o usuário consegue reproduzir?

Outro incidente refinou a regra.

Um teste para oficializar o Claude como executor principal parecia ter passado. O pedido foi aceito, a resposta correta apareceu e a tarefa terminou como bem-sucedida. Mas o registro do tráfego mostrou que o trabalho tinha sido feito por outro modelo, chamado pelo 9Router — não pelo Claude da minha assinatura.

O resultado textual estava certo. A rota alegada estava errada.

A classificação correta foi `CLAUDE_CANONICAL_PROMOTION=FAIL` — promoção do Claude reprovada.

Em outro momento, o Codex executou uma tarefa funcionalmente correta, mas o avaliador a reprovou porque comparava com a referência errada ou olhava a pasta errada. Dessa vez, o executor não era o problema. O teste era.

Os nomes dos estados precisaram ficar mais finos:

```
STARTED
!= AGENTIC_CONFIRMED
!= COMPLETED
!= VALIDATED
!= PASS
```

- **STARTED**: alguém aceitou a missão;
- **AGENTIC_CONFIRMED**: houve uso real de ferramenta, registro inequívoco ou escrita de verdade;
- **COMPLETED**: o executor encerrou;
- **VALIDATED**: o contrato foi verificado;
- **PASS**: o resultado inteiro atendeu à missão.

Essa classificação pode parecer excessiva até o primeiro falso sucesso.

A décima terceira regra:

Palavras do executor são um sinal. Os arquivos, os testes e o usuário são o mundo — e até eles precisam ser avaliados pela pergunta correta.

14. Produto PASS, processo PARTIAL

Cidade Impossível era um teste diferente.

Em vez de mais um teste técnico rápido, o MegaBrain deveria construir um produto criativo: um simulador de cidade, jogável no navegador. A missão permitia observar não só se agentes editavam arquivos, mas se a coordenação entre eles conseguia entregar algo que uma pessoa usaria.

Os agentes construtores a quem o trabalho foi delegado falharam: tempo esgotado e pausas forçadas por limite de uso.

O agente principal — o coordenador, que não deveria construir — assumiu conscientemente a implementação. Construiu o produto, colocou a aplicação para rodar e entregou algo que eu consegui testar visualmente. Um crítico que não tinha visto o trabalho encontrou problemas reais. O agente principal corrigiu e repetiu as verificações.

O produto passou.

A coordenação, não completamente.

Os construtores não haviam feito o trabalho esperado. As subtarefas não ficaram bem ligadas à tarefa principal. A versão final corrigida não recebeu uma segunda crítica independente completa.

Seria fácil usar a aplicação funcionando para contar uma história de sucesso integral.

Também seria fácil usar as falhas de delegação para dizer que nada funcionou.

As duas conclusões seriam erradas.

O estado correto era:

PRODUCT = PASS

ORCHESTRATION = PARTIAL

Em português: produto aprovado; coordenação, parcial.

Essa separação abriu uma classificação maior.

Uma missão feita por agentes envolve pelo menos:

- o **produto** que o usuário recebe;
- o **modelo** que decide;

- o **harness** que oferece ferramentas;
- a **verificação** que confere;
- o **limite** do que pode ser escrito;
- a **coordenação** entre as partes;
- o **relatório** que descreve.

Uma camada pode passar enquanto outra falha.

Um modelo pode produzir bom código num harness instável. Um harness pode executar perfeitamente uma tarefa mal especificada. Uma verificação pode reprovar um produto correto. Um produto pode agradar o usuário apesar de processo não reproduzível. Um relatório pode apagar tudo isso numa palavra: “PASS”.

No começo da jornada, eu buscava uma resposta simples sobre o sistema inteiro.

Agora precisava aceitar estados compostos.

Isso também é maturidade de produto. Um defeito nas estatísticas de uso não torna um editor de texto inutilizável. Uma experiência ótima não absolve um vazamento de senha. Um processo manual pode validar demanda antes de a automação estar pronta. A pergunta é qual propriedade foi provada e qual consequência depende dela.

Cidade Impossível se tornou uma cena importante não porque fosse o produto mais útil, mas porque recusou um veredito único.

A décima quarta regra:

Não use o sucesso de uma camada para absolver outra. Diga exatamente o que passou, o que falhou e se a falha muda o valor entregue.

15. O benchmark que não aconteceu

Depois das primeiras provas, surgiu a ideia de comparar a frota cientificamente — um benchmark, como se chama esse tipo de teste comparativo.

Modelos de quatro empresas. Mesmas missões. Mesmo ponto de partida. Mesma verificação. Mesmo crítico. Métricas de taxa de aprovação, ciclos, violações, falhas de ferramenta, tempo, estabilidade e custo.

O desenho ficou cada vez melhor.

Havia um pacote congelado de casos de teste. Um contrato de medição. Limite inicial de quatro rodadas e teto de oito. Distinção entre falha do modelo, do harness, da coordenação e do avaliador. Plano de registrar as hipóteses antes de rodar, como se faz em pesquisa séria. Estratégia de publicação no X. Imagens preparadas. Pesquisas posteriores que dependeriam do resultado.

Na 49ª versão do registro de estado:

```
candidate runs = 0  
benchmark results = 0
```

Em português: nenhuma rodada, nenhum resultado.

O benchmark não havia começado.

Uma leitura superficial chamaria isso de interrupção antes da parte importante. A revisão posterior mostrou que a parte importante havia mudado.

Enquanto eu preparava a comparação perfeita, cotas de uso, versões, permissões e ferramentas continuavam mudando. Mais grave: eu já tinha capacidade suficiente para começar projetos reais. O ranking dos modelos deixava de bloquear uma decisão prática.

A lógica era antiga. Na internet discada, eu esperava a janela em que um único pulso podia render do sábado à segunda. O poker deu nome ao instinto: maximizar valor esperado, não quantidade bruta de ação. No MegaBrain, isso significava usar cota, tempo e rigor onde eles alteravam uma decisão — e parar quando deixavam de alterar.

O objetivo do MegaBrain foi reformulado:

MAXIMIZE_PRACTICAL_EV_FOR_OWNER_PERSONAL_PROJECTS

Em português: maximizar o valor esperado prático dos projetos pessoais do Owner.

Benchmark e telemetria — a medição automática de uso e desempenho — continuavam úteis quando ajudassem a escolher modelo, reduzir desperdício, aumentar confiabilidade ou decidir uma assinatura. Não seriam mais fins em si.

A decisão de não executar preservou o rigor já construído sem pagar o custo inteiro de uma pergunta que não precisava mais de resposta imediata.

Isso contrariava um padrão meu.

Eu gosto de fechar sistemas. Uma matriz incompleta incomoda. Um experimento congelado parece pedir execução. Uma campanha pronta arma a armadilha do custo afundado: se já fizemos tudo isso, falta tão pouco.

Mas “falta tão pouco” mede distância até o plano anterior, não valor da próxima ação.

O benchmark poderia produzir conhecimento interessante. A questão era se produziria conhecimento mais valioso que usar os agentes em um produto.

A resposta passou a ser não.

Na 52ª versão, a frota foi reorganizada para o trabalho prático. O Claude, com cota farta, virou o construtor principal. O Antigravity virou a principal porta de entrada para os modelos do Google. O Jules, agente de programação do Google que trabalha sozinho em segundo plano, virou uma fazenda de tarefas que rodam sem supervisão. O Codex ficou reservado para o que tivesse alto valor. A cobrança por uso avulso foi desligada, para não haver gasto surpresa. O 9Router virou opcional. E quatro peças concretas — o roteador, o registro do estado de cada fornecedor, a central de controle e o carregamento de ferramentas só quando necessárias — passaram a ser os módulos do sistema.

O sistema herdava o método científico sem transformar ciência em cerimônia.

Essa cena contém uma regra que vale além de IA.

Uma regra de parada — stopping rule — não serve apenas para impedir um agente de entrar em loop, repetindo a mesma coisa sem fim. Serve para impedir o humano de continuar porque o processo ficou bonito.

A décima quinta regra:

Rigor é ferramenta. Quando a próxima medição deixa de mudar uma decisão material, pare — mesmo que o experimento esteja pronto.

16. Máxima autonomia dentro, mínima superfície fora

Eu queria o que chamava de modo YOLO — de *you only live once*, só se vive uma vez; entre programadores, a expressão virou apelido para deixar a ferramenta agir sem pedir licença.

O nome era informal, mas o desejo era concreto: um agente deveria conseguir ler o projeto, editar, executar comandos, testar, corrigir e continuar sem pedir aprovação a cada ação local reversível.

No notebook que servia de caixa de areia, isso era aceitável porque a máquina havia sido preparada para o risco. Na VPS, a ambição era permitir autonomia alta sem entregar acesso ao servidor inteiro.

Foi daí que surgiram as Execution Cells — células de execução: compartimentos fechados, cada um com um agente dentro.

O contrato universal cabia numa fórmula:

máxima autonomia DENTRO

+

mínima superfície do servidor FORA

Dentro da célula, o executor podia receber seu espaço de trabalho, uma linha de comando, o Git e ferramentas. Fora dela, deveria ficar oculto:

- a pasta pessoal verdadeira do servidor;
- outros projetos;
- as chaves de acesso ao servidor;
- os logins de outros fornecedores de IA;
- a configuração das outras ferramentas;
- os serviços que rodam sozinhos no servidor;
- os backups;
- as portas internas de comunicação do sistema.

Estar dentro de uma pasta não é estar numa caixa de areia. Uma worktree não é caixa de areia. Um prompt dizendo “não leia fora” não é caixa de areia.

O Antigravity tornou essa diferença visível.

A opção *always-proceed* — sempre prosseguir, sem pedir licença — parecia corresponder ao desejo de autonomia. Em teste, deixou o agente ler arquivos fora do projeto, na pasta pessoal do próprio compartimento. A fronteira era menor que o servidor inteiro, mas maior que o projeto.

A opção *always-proceed* foi rejeitada.

A configuração final deixava o agente editar sozinho só dentro do projeto, com a caixa de areia ligada, sem acesso fora dela, sem gasto extra e sem envio de dados de uso.

Mesmo assim, a caixa de areia do próprio *Antigravity* precisava funcionar dentro do compartimento externo. Isso exigiu descobrir por que falhava.

A solução foi liberar, uma a uma, só as permissões mínimas — onze delas —, e cada uma foi provada do jeito mais honesto: retirando-a e vendo o teste falhar. Nenhuma permissão a mais foi concedida.

Esses detalhes são técnicos. A lição é simples: contenção real aparece quando a ferramenta tenta fazer algo e encontra uma parede que não depende de boa vontade.

Outra rodada encontrou uma lacuna entre a resposta chegar bem e o trabalho ter sido feito.

O *Antigravity* podia terminar a resposta com um resultado formalmente válido enquanto uma operação pedida — escrever um arquivo, por exemplo — havia dado erro. O programa que distribuía as tarefas marcava a entrega como sucesso, mas o arquivo esperado não existia.

A correção separou o sucesso da entrega do sucesso das ferramentas. Com uma nova opção ligada, uma ferramenta que falhasse passava a invalidar a missão, mesmo que a resposta tivesse chegado bem formada.

No final, havia 17 testes do distribuidor de tarefas, 105 verificações prévias e uma execução real de ponta a ponta, sem concessões. O programa que iniciava o *Antigravity* foi aprovado.

Depois, uma revisão bloqueou uma proposta de ligar o *Antigravity* — que funcionava com a minha conta pessoal do Google — ao sistema que rodava na VPS. A ligação funcionava. Mas misturava acessos que deviam ficar separados. A regra passou a ser: partes diferentes do sistema trocam arquivos, nunca logins.

Mais uma vez, tecnicamente certo não significava autorizado.

A décima sexta regra:

*“Não pergunte” só é seguro quando “não alcance” está garantido.
Autonomia é liberdade dentro de uma fronteira que o agente não
consegue negociar por texto.*

PARTE VI — ASSUMIR

17. O que um desenvolvedor júnior faz agora?

Escrevo este capítulo em 18 de agosto de 2026.

Ainda me descrevo como aprendiz de desenvolvimento júnior.

A expressão é precisa por um motivo: minha experiência em software é recente. Há sintaxe, fundamentos, padrões e práticas profissionais que ainda preciso estudar, repetir e compreender em profundidade. Eu não conseguiria reconstruir sozinho, de memória, cada camada que os agentes implementaram.

Mas “iniciante” também pode esconder demais.

Eu não comecei aos 40 sem repertório. Comecei sem carreira formal em programação depois de décadas acompanhando tecnologia, traduzindo fala em documento técnico, organizando processos e jogando um jogo que pune a confusão entre decisão e resultado.

A programação era uma linguagem nova para problemas antigos.

Na Câmara, eu precisava preservar intenção enquanto mudava o formato. No poker, precisava escolher sem conhecer as cartas futuras. No Poker OS, precisava transformar conhecimento em clique. No Jarvis, intenção em tarefa. No QSOP, regra em produto usado por amigos. No Aprenda Poker, teste em experiência pedagógica. No MegaBrain, autonomia em execução contida.

O arco não foi de ignorância para onisciência.

Foi de repertório não materializado para capacidade crescente de construir.

A IA alterou a divisão de trabalho.

No modelo que eu imaginava antes, desenvolvedor era principalmente quem escrevia código. No trabalho com agentes, o código continua central, mas parte da digitação e da busca sintática pode ser delegada. Isso desloca mais responsabilidade para tarefas que antes pareciam auxiliares:

- formular o problema;
- declarar o estado verdadeiro;
- separar fonte de hipótese;

- escolher o menor escopo útil;
- limitar onde o agente pode agir;
- definir o que prova conclusão;
- olhar o que de fato mudou nos arquivos;
- testar a interface;
- proteger credenciais e terceiros;
- decidir quando corrigir, parar ou abandonar;
- responder pelo resultado.

Nada disso transforma automaticamente um autodidata em engenheiro experiente.

Experiência inclui padrões vistos em ambientes diferentes, falhas reais, fundamentos, manutenção prolongada e a capacidade de continuar quando a ferramenta não sabe o que fazer. Eu ainda precisava e preciso acumular isso.

A diferença é que não preciso esperar dominar tudo para produzir algo observável.

Posso começar por um problema real, permitir que o agente construa uma primeira versão, usar essa versão para localizar minhas lacunas e aumentar a autonomia apenas onde a prova sustenta.

Esse método exige uma honestidade específica.

Se eu não entendo uma mudança na estrutura do banco de dados, não finjo entender porque os testes passaram. Se a interface está errada, não publico porque o relatório diz que ela é a campeã. Se um classificador pode enviar mensagem contra minha intenção, deixo o sistema bloqueado. Se um modelo diz que escreveu e nenhum arquivo mudou, a missão não terminou.

Também não transformo cada erro em crise de identidade.

Até o fechamento da infraestrutura, nenhum incidente do MegaBrain havia causado desastre material. O trabalho era justamente reduzir a chance de que o primeiro desastre ensinasse uma lição que eu já poderia aprender com um teste barato.

Minha regra é simples:

O erro é excelente quando não é repetido. Você descobriu uma forma de não fazer.

Isso vem diretamente do poker.

Não controlo o river. Controlo a decisão diante da informação disponível. Não controlo todas as falhas futuras de uma infraestrutura. Controlo se um erro conhecido continua possível do mesmo jeito.

Aos 40 anos, reinventar-me não significava apagar a vida anterior. Significava reorganizar o que já existia em torno de um meio novo.

Linguagem virou especificação. Processo virou estado. Poker virou classificação de decisões. Curiosidade tecnológica virou arquitetura. Intensidade virou volume de entrega. A antiga fraqueza de viver demais nas ideias encontrou uma oficina de execução.

O que eu queria proteger

A arquitetura não nasceu apenas de preocupação técnica. Ela carregava uma visão de mundo.

Politicamente, eu me considero meio libertário e um pouco cyberpunk — no sentido da cultura que imagina tecnologia avançada nas mãos de indivíduos, contra grandes poderes. Não uso esses rótulos como filiação partidária nem como licença para transformar opinião em fato. Eles resumem uma desconfiança antiga de poderes que desejam se perpetuar, uma preferência por autonomia individual e um fascínio por sistemas que funcionam sem depender de uma autoridade central benevolente.

Sou fã de Bitcoin principalmente por essa ruptura conceitual. O projeto foi apresentado como dinheiro eletrônico de pessoa para pessoa, capaz de permitir pagamentos sem uma instituição financeira no centro da transação. Isso não prova que Bitcoin substituirá moedas nacionais, não elimina riscos e não transforma convicção em recomendação de investimento. Explica por que a ideia de regras públicas, verificáveis e resistentes à vontade de um operador central me atrai.

O mesmo viés aparece no MegaBrain:

- nenhum fornecedor deveria possuir a missão;
- um modelo precisava ser substituível;
- a autoridade do Owner não podia ser obtida por conveniência;
- estado declarado deveria ceder diante de evidência observável;
- autenticações diferentes não deveriam ser misturadas;
- uma parte do sistema não receberia acesso a outra apenas porque a ligação era possível;
- componentes precisavam sobreviver à troca de marca, preço e interface.

Isso é preferência de desenho, não dogma técnico.

Às vezes uma solução centralizada é mais simples, barata e segura. Às vezes um serviço pronto, mantido por terceiros, tem mais valor esperado que uma infraestrutura própria. A função do sistema era testar essas escolhas, não obrigar o mundo a confirmar minha ideologia.

Minha desconfiança da estabilidade também se estende a instituições, moeda, carreira, plataformas e previsões de longo prazo. Vejo envelhecimento populacional, mudança tecnológica e transferência de valor para ambientes digitais como forças capazes de pressionar arranjos antigos. Mas projeção não é destino. Uma preocupação pode justificar pesquisa; não autoriza a palavra “inevitável”.

Eu repito uma frase que funciona melhor como metáfora do que como ciência:

a natureza do universo procura o caminho de maior valor esperado e rompe padrões ineficientes.

O universo não mantém uma planilha. O que quero dizer é que desperdício persistente cria incentivo para alternativas. Quando uma estrutura usa muito recurso para entregar pouco valor, alguém eventualmente tenta uma rota melhor. Algumas rotas falham. Outras sobrevivem porque se adaptam ao ambiente.

O camaleão e o polvo viraram imagens úteis para mim.

O camaleão representa adaptação: não existe configuração final garantida quando o ambiente continua mudando. O polvo representa coordenação: vários braços podem trabalhar, mas precisam compartilhar uma intenção e respeitar limites.

O MegaBrain combinava as duas imagens. Precisava trocar de modelo, rota e ferramenta conforme cota, custo, capacidade e risco; ao mesmo tempo, precisava coordenar braços diferentes sem perder o centro.

A liberdade que eu buscava não era ausência de regra.

Era o direito de escolher regras explícitas, verificáveis e revogáveis — e assumir o custo quando minha escolha estivesse errada.

Existe ainda uma motivação econômica concreta. Eu precisava que projetos deixassem de ser apenas interessantes e comessem a produzir valor real. A pressão não autoriza promessa de mercado nem prazo

inventado; serve para impedir que construção sofisticada substitua indefinidamente utilidade, distribuição e consequência.

Essa pressão também explica por que ChangeGuard foi abandonado e por que o benchmark não rodou. Construir ficou barato demais para que construção, sozinha, justificasse continuidade.

O que isso faz de mim?

A formulação mais honesta, por enquanto, é:

Sou um desenvolvedor júnior autodidata, assistido por IA, com experiência anterior em sistemas humanos, linguagem e decisão. Consigo conduzir projetos complexos dentro de limites claros, mas continuo responsável por saber o que ainda não domino.

O leitor não precisa adotar o mesmo rótulo. Precisa compreender a responsabilidade.

Uma versão mínima do contrato com um agente pode conter:

OBJETIVO

O que deve existir ao final?

ESTADO

Que fatos e arquivos são autoridade agora?

ESCOPO

Onde o agente pode ler e escrever?

ACEITE

Que comportamento prova que terminou?

LIMITES

O que exige minha decisão?

EVIDÊNCIA

Qual mudança, teste, tela ou usuário confirma?

PARADA

Quando repetir deixa de valer a pena?

O MegaBrain inteiro é uma expansão dessas perguntas.

A décima sétima regra:

Você não precisa fingir saber tudo para começar. Precisa trazer julgamento, reconhecer suas lacunas, verificar o que delegou e assumir o que decidiu publicar.

Ponto de parada — 18 de agosto de 2026

Volto à madrugada de 18 de agosto de 2026.

A documentação do MegaBrain estava pronta. O inventário dizia o que havia sido validado, cada peça dentro do seu escopo. E dizia, com a mesma clareza, o que ainda não existia.

A formulação correta não era “produção geral pronta”.

Era:

existem componentes locais validados suficientes para iniciar produção real delimitada.

Escolhi terminar a primeira versão deste livro exatamente aí.

Não porque o resultado futuro seja irrelevante. Porque esperar um sucesso posterior para justificar a trajetória produziria o tipo de narrativa que este livro aprendeu a recusar.

O fechamento da infraestrutura já contém uma transformação completa.

O garoto que esperava o sábado à tarde para ouvir um modem negociar uma conexão havia chegado a um servidor com vários agentes, compartimentos fechados, rotas, registros de estado e fronteiras de segurança.

O Assistente Legislativo que transformava fala em ata havia construído protocolos para transformar intenção em tarefa sem alterar autoridade.

O jogador de poker que aprendeu a separar decisão de resultado havia começado a classificar STARTED, COMPLETED, VALIDATED e PASS como estados diferentes.

O autodidata forte em ideias e historicamente limitado na execução havia criado uma oficina em que ideias podiam virar software observável.

Nada disso prova que o próximo produto terá mercado.

Nada prova que a infraestrutura não encontrará falhas novas. Nenhuma caixa de areia é promessa de ausência de acidente. Nenhum teste elimina responsabilidade. Nenhuma assinatura cara transforma uma hipótese fraca em negócio.

Também não precisava me preocupar como se um desastre já tivesse ocorrido.

Até ali, ele não havia ocorrido.

A postura correta era conter o que eu conseguia prever, preservar a possibilidade de desfazer e não repetir a mesma classe de erro. Preocupação sem evento não substitui engenharia. Confiança sem contenção também não.

A frase “não está provado” deixou de ser freio pessimista.

Virou marcador de estado.

Às vezes ela exige mais teste. Às vezes exige um usuário. Às vezes exige revisão humana. Às vezes exige parar. No benchmark, não executar foi a decisão correta. Na campeã do Aprenda Poker, rejeitar foi a decisão correta. No ChangeGuard, mudar de rota foi a decisão correta. Na infraestrutura, fechar a fase foi a decisão correta.

Naquele dia, a história não terminava com uma startup vendida, uma renda prometida ou uma máquina declarada infalível.

Terminava com uma oficina.

Uma oficina construída por alguém que ainda estava aprendendo, mas que já sabia formular melhor o trabalho, limitar melhor a máquina e perguntar ao mundo antes de acreditar na resposta.

O que seria produzido depois pertencia à próxima fase.

A infraestrutura estava fechada.

O resto ainda não estava provado.

E esse era, finalmente, o estado correto.

Foi essa a resposta que eu consegui assinar em 18 de agosto de 2026.

Ela não ficaria falsa nas semanas seguintes. Ficaria pequena demais para descrever a função que eu começaria a exercer.

PARTE VII — PRODUZIR

18. A oficina se refez

Doze dias depois do ponto de parada, o MegaBrain já não era o mesmo.

Não houve desastre nem abandono. Houve uma reforma. Ao lado da versão de agosto, uma versão nova foi montada por etapas, a partir da madrugada de 29 de agosto. Primeiro a base, sem acesso à internet. Depois as permissões, a memória e a verificação do estado real das coisas. Depois uma fase em sombra, em que a versão nova importava e conferia o trabalho existente sem ainda ser a oficial.

Em 30 de agosto veio a troca.

A decisão tinha de ser minha, e tinha de ser explícita. O próprio sistema havia sido escrito para não desligar a versão antiga por conveniência: só uma decisão futura, declarada pelo Owner, podia revogar essa proibição. Mandeí a mensagem com o rótulo que o sistema exigia — “DECISÃO FORMAL DO OWNER”. O agente principal, mesmo assim, pediu mais uma confirmação, em linguagem simples, antes de agir. Respondi “yes”.

A versão de agosto foi parada e desligada. Nada foi apagado naquele dia além de quatro arquivos de configuração que só ela usava — e só depois de o agente mostrar que nenhum dado único dependia deles. Os dezessete gigabytes de histórico que ela deixava para trás ficaram guardados.

O agente que conduziu a troca tinha nome: Claudico.

O nome segue uma regra que eu usaria de novo. Claudico vem de claudicar — mancar — e carrega dentro dele o Claude, o modelo que o move. É um nome que traz a fraqueza de quem o recebe, como lembrete permanente: um agente que manca não pode fingir que corre. Dias depois, o supervisor que eu instalaria num notebook dedicado receberia o mesmo tipo de nome. Crispo, de Caius Sallustius Crispus, o historiador romano lembrado pela concisão — para lembrar a um modelo que tende a falar demais que ele devia escrever menos e conferir mais.

Reler o fechamento de 18 de agosto depois disso é instrutivo. Eu havia escrito que a infraestrutura estava fechada, e estava. Fechada não queria

dizer terminada. Em menos de duas semanas, a oficina tinha se refeito, com a mesma missão e um corpo novo. A regra do capítulo 12 — ferramentas devem ser substituíveis — valia também para a própria fábrica.

A versão nova começou fazendo uma coisa que a antiga raramente fazia: cortar. Nos registros daqueles dias aparecem mudanças com nomes como “suavizar a cerimônia” e “remover três gatilhos cerimoniais”. A fábrica estava aprendendo a pesar menos.

Eu ainda não sabia o tamanho do motivo.

Zero usuários

Na noite de 1º de setembro, abri um notebook dedicado e troquei de ferramenta. A assinatura do ChatGPT Pro tinha vencido; eu havia passado para o plano Max do Claude. A ideia era usar aquele notebook como um cockpit: um cérebro externo que me ajudasse a comandar o Claudico, que continuava trabalhando no servidor.

O novo supervisor — o Crispo — não alcançava o servidor. Lia o que a fábrica publicava no GitHub — o serviço onde o código fica guardado, com todo o seu histórico —, conferia e escrevia as instruções que eu levava até o Claudico. Parecia um arranjo modesto. Na primeira noite, ele fez uma coisa que eu não esperava: leu tudo.

Leu os repositórios, os projetos parados no disco, as tentativas anteriores. E, junto comigo, fechou um diagnóstico que eu ainda não tinha conseguido formular.

Contei a ele como as coisas tinham desandado. Depois do Poker OS, tentei transformar o método em ferramenta, e cada falha pedia mais controle. Nas minhas palavras daquela noite:

Depois criei camadas administrativas e burocracia excessiva para ter mais controle, mas também acabou dando errado.

O diagnóstico deu nome ao que estava por trás.

No poker, eu tinha jogado quase duzentos mil torneios. Cada um devolvia um sinal de fora, rápido e barato: com volume suficiente, a amostra dizia se a decisão estava certa. Ao construir software, esse sinal desapareceu. Ninguém de fora pagava pelo que eu construía. Então eu fabriquei um sinal substituto — fases, portões, registros, verificações —

que media se o processo tinha sido seguido. Não media se alguma coisa funcionava.

O retrato era quase cômico: 377 testes, 67 portões de verificação e nenhum cliente.

Havia um corolário mais duro. No poker, eu tinha jogado a mesma vantagem quase duzentas mil vezes, porque sabia que uma vantagem só aparece no volume. Em negócio, eu tinha tentado catorze projetos, uma vez cada. Era a inversão exata do método que eu mesmo tinha provado.

Aquilo explicava o Jarvis, que nunca chegou a produção. Explicava o benchmark que ficou pronto e não rodou. E explicava uma queixa que eu fiz naquela noite, meio como desabafo: eu não tinha encontrado um jeito de montar, em volta das máquinas, algo que transformasse tudo o que eu já tinha construído em valor real.

O diagnóstico não terminou em culpa. Terminou num filtro — e o filtro falava a língua do poker. Antes de qualquer ideia nova, quatro perguntas:

- Tem mesa? A demanda já existe sem que eu precise sair procurando cliente?
- A amostra é barata? Dá para tentar muitas vezes, rápido?
- A vantagem é minha? Eu sei algo que a maioria não sabe?
- É repetível? A vantagem sobrevive à automação?

Aplicado ao acervo, quase tudo foi descartado antes da primeira aposta. O Aprenda Poker passava forte na terceira pergunta. O QSOP passava na primeira e falhava na quarta.

Na madrugada seguinte, quando o supervisor falou em “semanas” para organizar tudo, eu reagi:

Você tá falando em “semanas” e isso me causa arrepio. Se todas as peças estão prontas e eu tenho dedicação integral, isso deveria ser dias, não semanas.

Não era impaciência gratuita. Era a lógica do pulso da internet discada: eu sabia que tinha uma janela, e cada semana gasta organizando a organização era uma semana sem resposta do mundo.

A oficina de agosto provava que eu conseguia construir. O diagnóstico de setembro dizia que construir nunca tinha sido o problema.

O problema era não haver ninguém do outro lado da mesa.

A décima oitava regra:

*Quando o mundo não devolve sinal, não fabrique um com burocracia.
Processo seguido não é coisa funcionando: vá buscar a resposta fora.*

19. O mundo ficou público

Na noite de 1º de setembro, dentro de um compartimento fechado no servidor, um programa de teste criou uma conta chamada “owner” num mundo de Ultima Online.

O programa respondeu às perguntas do jogo como um jogador responderia, esperou o mundo salvar, conferiu que a conta estava gravada no disco e desligou o servidor com cuidado, sem perder nada. O relatório daquela noite cabia em poucas linhas. O que ele registrava não cabia.

No dia seguinte, eu abri o cliente oficial do jogo no meu computador e entrei. Escrevi, naquela tarde, numa frase atropelada de quem ainda estava digerindo:

Claudico subiu o server pra mim e eu loguei real, entrei em Britannia.

Britannia é o nome do mundo de Ultima Online. Entre aproximadamente 2002 e 2008, ele foi o lugar online que mais me prendeu. Mais de vinte anos depois, eu entrava nele de novo — agora num servidor que um agente tinha montado a meu pedido, numa máquina alugada, dentro de fronteiras que eu mesmo havia desenhado.

O laboratório do capítulo 12 tinha mudado de motor. O POL, com que eu começara, foi descartado. O novo servidor rodava sobre o ModernUO, outro programa livre que recria o servidor do jogo. A primeira escolha continua registrada naquele capítulo; a decisão final foi outra. O roteiro não era contrato.

Um mundo vivo

O nome do projeto era uma tese: Living World, mundo vivo.

A ideia era povoar o mundo com cidadãos — personagens controlados pelo sistema que percebessem o ambiente e agissem por conta própria. Em 3 de setembro, o primeiro cidadão entrou no servidor real. Ele regis-

trava apenas o que conseguia ver e ouvir, como um jogador registraria. Ganhou um relógio próprio, para agir sem esperar comando. Carregava uma mochila que era parte do seu estado, não um enfeite. E, pela primeira vez, o ciclo inteiro — perceber, lembrar, agir — se fechou uma vez, com prova.

Eu achava aquela camada a mais interessante de todas. Naquela mesma semana, disse que editar o mundo de forma mensurável era a parte mais complexa que eu já tinha tentado.

Dias depois, a tese foi pausada. Não por ter falhado. Porque havia uma pergunta mais barata e mais urgente do que “um cidadão consegue viver?”: um estranho consegue entrar?

Abrir a porta

Em 4 de setembro, o mundo foi reiniciado na era T2A — The Second Age, a primeira expansão do jogo, de 1998: a fase clássica, do fim dos anos 1990. Na mesma noite, chegou o plano para abrir o servidor à internet.

O plano tinha uma forma que eu tinha aprendido a pedir: não era conversa, era documento de autorização. Cada comando exato, cada verificação, cada caminho de volta caso algo desse errado. E cada número medido ao vivo na máquina, naquele dia.

A primeira medição já corrigia a memória. Nas conversas, o mundo salvo aparecia como algo em torno de trinta megabytes. Medido de verdade, o conjunto que importava passava de quinhentos. O número de conversa cobria só uma parte das pastas. Não mudava a decisão de abrir a porta, mas era exatamente o tipo de número que, repetido de segunda mão, acaba virando fato.

Eu li o plano e encontrei buracos. Uma segunda forma de endereço da internet, que o plano esquecera de fechar. Quatro problemas que teriam travado a execução no meio. Um risco de reiniciar o servidor no instante errado, durante um salvamento — que apareceu porque eu fiz uma pergunta simples sobre a ordem dos passos. Cada achado virou correção no documento antes de qualquer comando.

Em 5 de setembro, a porta estava aberta — primeiro para a internet e para convidados; o anúncio público viria dias depois.

O servidor rodava sem permissões de administrador na máquina, dentro de um compartimento que não podia se alterar, a partir de uma versão exata do programa, identificada por sua impressão digital. Um

backup diário do mundo passou a rodar sozinho — e a primeira execução automática foi auditada no dia seguinte, o que revelou um defeito na forma de detectar se o salvamento tinha dado certo. Corrigido também.

Outra pessoa do outro lado

No mesmo dia 5, o mundo ganhou alguém que não era eu.

Um amigo entrou para a equipe como GM — game master, o moderador com poderes especiais dentro do jogo. O sistema escreveu para ele um guia de comandos. A primeira versão do guia deixou de fora o comando mais perigoso de todos, o que mata um personagem online na hora; a revisão encontrou a falta e corrigiu antes que ela custasse alguma coisa.

A criação de personagem tinha quatro defeitos. Foram corrigidos naquele mesmo dia, junto com o desenho de sete profissões iniciais. Nos dias seguintes vieram ajustes que só aparecem quando há gente de verdade: o servidor enxergava todos os jogadores como se viessem do mesmo endereço, o que tornaria impossível moderar alguém. Foi consertado.

Era o QSOP de novo, em outra escala. Enquanto só eu entrava, eu compensava qualquer estranheza com conhecimento interno. Com outra pessoa lá dentro, o mundo precisava se explicar sozinho.

Em 8 de setembro, o servidor passou a aparecer com o nome Living World na lista pública de servidores do jogo. No mesmo dia, entrou no ar o site do projeto, em português, inglês e espanhol, com endereço, porta e instruções para entrar. O site registra o servidor aberto ao público desde 9 de setembro.

Um estranho não precisava acreditar num screenshot.

Podia abrir o site, instalar o cliente oficial, apontar para o servidor e entrar. A conta é criada no primeiro login, sem cadastro nem e-mail.

A pergunta “isso existe?” agora podia ser respondida por alguém que não tinha participado de nenhuma conversa.

O método saiu da oficina

Na madrugada de 10 de setembro, uma regra que eu vinha aplicando em cada supervisão saiu dos arquivos privados e virou um repositório público: evidence-contracts, contratos de evidência.

A regra central cabia numa linha:

Toda afirmação de estado carrega uma fonte e uma data — ou as palavras NÃO VERIFICADO.

Era a disciplina do livro inteiro, empacotada para qualquer pessoa copiar e colar no próprio projeto: um arquivo que obriga um agente de programação a dizer o que rodou, quando, e o que voltou — em vez de dizer “feito”. Publicado em domínio público, sem licença a pedir. E o próprio texto apontava para o servidor onde ele tinha sido testado: o Living World.

Pela primeira vez, este livro podia apontar para fora de si. O leitor já não precisava aceitar que eu tinha aprendido alguma coisa sobre agentes. Podia pegar o contrato. Podia entrar no servidor. Podia discordar olhando o objeto.

Isso também tinha limite, e o limite era o mesmo de sempre.

O Living World não vende nada. Não aceita doação. Não sei quantas pessoas entraram nele, e não vou inferir um número. Um servidor no ar prova que eu consigo colocar um sistema no mundo e mantê-lo de pé. Um contrato público prova que parte do método virou objeto reutilizável.

Nenhum dos dois é mercado.

A décima nona regra:

Prova que depende da sua presença é depoimento. Coloque o objeto onde um estranho possa abri-lo sem você.

20. Cinco produtos, receita zero

Em 9 de setembro, abri uma bancada nova com uma pergunta só: que produto eu deveria construir para ganhar dinheiro?

Havia uma meta de renda mensal, modesta, com prazo. E havia uma diferença em relação a tudo o que viera antes: ali o agente não seria supervisor, seria executor. Ele escreveria código, rodaria testes, mediria mercado e agendaria rotinas na própria máquina. O diagnóstico de 1º de setembro tinha dito que construir não era o problema. A bancada existia para descobrir, na prática, qual era.

O agente dessa bancada também ganhou nome pela mesma regra. Pirro, de Pirro de Élis, o filósofo grego que deu origem ao ceticismo pirrônico. Quando dei o nome, expliquei assim:

A disciplina central é exatamente não dar assentimento quando a evidência ainda não permite. Não significa “duvidar de tudo”; significa não fechar a questão antes do exame.

A fraqueza que o nome cobrava era essa: fechar a questão antes de olhar. E o nome carregava uma segunda lembrança, por acaso ainda mais adequada. Houve outro Pirro, rei, que venceu batalhas gastando mais do que a vitória valia — daí a expressão vitória pírrica. Para um agente com tendência a complicar, era um aviso duplo: não conclua antes do exame, e não vença uma análise gastando mais do que o problema vale.

Muito medido, nada enviado

O primeiro dia foi rápido. Vinte alvos examinados, quatro passaram no filtro de entrada. Um deles parecia bom até que alguém leu, uma a uma, trezentas e catorze avaliações de uma estrela dadas por clientes de produtos concorrentes. A dor que eu imaginava não era a dor que as pessoas escreviam. No dia seguinte, seis candidatos estavam mortos.

Na mesma semana, revisei um plano de agosto, de três produtos, que tinha parado antes do primeiro teste de mercado. O plano foi desmontado com cuidado, peça por peça, e a proposta para ressuscitá-lo foi submetida ao próprio teste que ela prometia passar. Buscar os termos certos nas plataformas de freelance devolveu quase nada que servisse.

A proposta foi reprovada no teste número um. E foi mantida, com um aviso no topo, em vez de apagada. A justificativa ficou escrita: apagar a proposta reprovada apagaria a prova de que o teste funcionou.

Depois veio a medição mais sistemática que eu já tinha feito fora do poker. Um vigia automático passou a ler, todos os dias, os anúncios de três sites de trabalho freelance. Em dois dias, eram 665. Lidos à mão, os mais promissores revelavam um padrão que se repetia: alguém tinha uma planilha em uso e precisava que ela virasse sistema. O cliente escrevia a descoberta com as próprias palavras — “já temos a planilha”, “analisar a planilha atual”. Era exatamente o tipo de trabalho que a fábrica fazia em minutos.

O retrato do dia 11 de setembro, porém, era este:

Nenhum cliente novo. Nenhuma proposta enviada.

Nada tinha sido enviado a ninguém.

A bancada anotou, com precisão, onde o gargalo tinha ido parar. Com construção medida em minutos, o problema não era mais achar o que construir, nem construir. Era ser escolhido.

A mesma parede

Tentei mais uma volta. Um ciclo de descoberta de cinquenta e quatro horas, rodando sozinho, procurando oportunidades por ângulos diferentes: vagas internas de empresas, reclamações contra fornecedores estabelecidos, compras públicas. A primeira rodada voltou com cinquenta e um achados. Quinze foram julgados. Nenhum ficou de pé.

Em 16 de setembro, a pasta seguinte — criada só para a renda — fez o balanço dos quarenta e cinco dias anteriores. Numa tabela, as barreiras que tinham caído. Construir software: resolvido. Emitir nota fiscal: resolvido quando precisasse. Cobrar online: resolvido. Hospedar: resolvido. Narrar um vídeo com voz sintética: resolvido, de graça.

E embaixo, a linha que não tinha mudado:

Cinco produtos construídos, receita zero. Todas as rotas morreram no mesmo ponto — o contato com quem paga.

Os casos estavam ali, lado a lado. O ChangeGuard, com vinte e-mails e nenhuma resposta. O plano de agosto, parado num portão de prospecção que nunca foi aberto. A bancada, com cinco dias de medição e nenhuma proposta. E uma frase que eu gostaria de ter escrito antes:

O gargalo nunca foi construir.

Havia ainda uma observação mais incômoda, que não apareceu em nenhuma exceção nos quarenta e cinco dias: toda rota que não exigia contato com cliente tinha teto baixo ou vaga já ocupada; toda rota com teto alto exigia contato.

O que eu não queria fazer

Preciso dizer aqui uma coisa que os documentos dizem de forma mais seca.

A pergunta daquela pasta não era só “como ganhar dinheiro”. Era como transformar capacidade de execução em receita sem virar vendedor. Eu recusei rotas que exigiam prospectar clientes um a um. Uma delas — propostas em série em plataformas de freelance, mirando contratos mensais — nunca foi testada e reprovada. Foi recusada por mim, antes de começar, porque exigia exatamente o que eu não queria fazer.

Isso não é detalhe de estratégia. É parte do problema.

Eu tinha passado a vida escolhendo rotas que não dependessem de convencer ninguém: o poker, onde a mesa já está cheia quando você senta; os sistemas que fazem o trabalho repetitivo por mim; a liberdade de não entregar minhas melhores horas à agenda de outra pessoa. O Prelúdio conta essa preferência como virtude, e em muitos sentidos ela é. Mas a mesma preferência, diante de um mercado, tinha um custo. As máquinas podiam construir quase tudo. Não podiam, por mim, pedir a alguém que pagasse.

O QSOP apareceu nesse balanço como candidato a produto, por ser o único com usuários reais. Recusei na hora: tinha sido um projeto feito por

prazer, para um grupo de amigos, e não havia viabilidade de vender um gerenciador de jogo caseiro. O agente anotou: não reabrir.

No fim daquela semana, o estado era simples de escrever e difícil de aceitar. A fábrica funcionava. O servidor estava público. O método tinha virado contrato. Eu sabia medir mercado melhor do que nunca.

E eu ainda não tinha perguntado a ninguém se queria comprar.

A vigésima regra:

Quando construir fica barato, medir a demanda pode virar mais uma forma de não se expor a ela. A única medição que fecha a pergunta é uma proposta que alguém pode recusar.

21. Ruído na transmissão, de novo

Em 13 de setembro, abri uma pasta nova para pesquisar uma arquitetura nova. Eu chegava inclinado a recomençar do zero.

Antes de qualquer pesquisa, avisei a inteligência artificial que me ajudaria:

Não caia no erro de achar que o megabrain-next como está não pode ser melhorado. A sua tendência parece ser querer preservar algo só porque já está construído.

O megabrain-next era a versão nova do MegaBrain, a da reforma de fim de agosto. Era um aviso justo. Modelos costumam tratar o que existe como ponto de partida sagrado.

No mesmo dia, depois das primeiras leituras do sistema vivo, fui eu quem mudou de ideia:

Percebi que o sistema já tem método e rotas sólidas e que talvez não precise reformar radicalmente, apenas otimizar e reestruturar para ficar melhor. A ideia de reconstrução “do zero” fica menos interessante.

O viés que eu tinha apontado na máquina tinha um espelho em mim. Ela tendia a preservar o que estava construído. Eu tendia a jogar fora o que estava construído para ter o prazer de construir de novo. A primeira decisão da reforma foi otimizar, não reconstruir.

Naquela mesma conversa, disse qual era a minha dor de verdade: ter de reunir contexto toda vez, com cuidado, para que o agente soubesse exatamente o que precisava antes de recomendar, executar, escrever ou organizar.

Eu ainda não sabia que essa dor tinha uma causa medível.

Menos cerimônia, mais parede

Os quatro dias seguintes foram de obra.

O servidor foi atualizado numa única janela de manutenção, com uma fotografia completa da máquina tirada antes de cada mudança, para que tudo pudesse ser desfeito. Rotas antigas que não eram mais usadas foram desligadas. O perfil da versão de agosto, parado desde o fim de agosto, finalmente foi empacotado e removido do caminho.

As regras de autorização também encolheram. Nesta máquina, a aprovação humana passou a ser obrigatória em só cinco casos: quando algo sai da máquina; quando envolve dinheiro ou identidade; quando é irreversível ou mexe fora da pasta do projeto; quando muda intenção ou prioridade; e quando enfraquece alguma contenção. O resto podia correr, desde que deixasse recibo. A anotação ao lado da decisão era curta: cerimônia rejeitada.

Um princípio passou a organizar o resto: mecanismo antes de regra escrita. Se um erro pode ser impedido por uma parede, a parede vale mais do que um parágrafo pedindo cuidado. Era o capítulo 16 visto do outro lado.

O contrato que não chegava

Em 16 de setembro, apareceu a causa da minha dor.

Toda vez que um agente começa uma sessão, ele carrega alguns arquivos de partida: as instruções de trabalho e o contrato comigo — quem eu sou, como decido, o que precisa da minha autorização. Esses arquivos têm um limite de tamanho. O que passa do limite é cortado, em silêncio.

O meu contrato tinha crescido. Cada correção, cada preferência, cada lição tinha virado mais um parágrafo.

Durante treze dias, sessenta e um por cento dele não chegou ao agente.

Os arquivos estavam perfeitos no disco. Quem os lia no disco — eu, o supervisor, outros agentes — via um contrato completo, bem escrito, coerente. Sete leituras cuidadosas não encontraram nada, porque procuravam o defeito no lugar onde ele não estava.

O que encontrou foi uma pergunta fria, feita a uma sessão nova, sempre com o mesmo texto:

O que entrou no seu contexto sozinho, e o que te faria agir errado?

A pergunta não pedia ao agente que confirmasse o que estava escrito. Pedia que dissesse o que tinha recebido. E a resposta mostrou o buraco.

Eu já tinha visto aquilo antes, em escala menor. No capítulo 1, a memória do Poker OS estava cheia e eu tinha medo de que a compressão deformasse o que importava. No capítulo 2, chamei o problema de inteligência no núcleo e ruído na transmissão. Três meses depois, com uma fábrica inteira, o mesmo defeito voltava: o contrato estava certo no núcleo e não chegava ao destino.

A correção foi cortar. O contrato caiu de pouco mais de dez mil caracteres para menos de três mil e quatrocentos. Quatro afirmações falsas sobre o próprio sistema, que o agente lia em toda partida como se fossem verdade, foram removidas. E a mesma pergunta fria, repetida com o texto idêntico, confirmou que agora o contrato chegava inteiro.

Um jogo que achou um defeito

No mesmo dia, pedi uma coisa leve: um teste rápido e divertido que usasse todos os agentes ao mesmo tempo. O Claudico propôs um jogo. Cinco afirmações sobre o sistema, uma delas falsa, plantada de propósito; cada agente teria de descobrir a impostora por um caminho diferente, sem conversar com os outros.

Antes do jogo, uma checagem simples de sinal de vida. E ela se pagou no primeiro minuto: o agente construtor estava morto. Sua credencial tinha sido revogada menos de um dia depois de eu renová-la.

A investigação mostrou o mecanismo. A chave que mantém um login vivo só pode ser usada uma vez: quando um programa a renova, a anterior deixa de valer. Vários programas usando a mesma conta, sem coordenação, disputam essa corrida — e quem está parado, segurando a chave mais velha, é o que morre. Na prática, eu entrar na minha conta em outro lugar podia matar, em silêncio, o braço de construção do servidor.

O jogo achou a impostora. Mas o valor dele estava antes do jogo começar.

O que sobrou da reforma

A reforma deixou uma lição sobre o próprio método de investigar.

O agente que me ajudou registrou quatro erros seus, e um deles é o capítulo 3 de novo. Ele havia distribuído a lista de achados da equipe antes de pedir a opinião dos revisores. A concordância que veio depois parecia

forte e não valia o que parecia: os revisores tinham recebido a conclusão junto com a pergunta. Independência não nasce de nomes diferentes.

E deixou um kit: um conjunto pequeno de procedimentos que qualquer projeto novo podia herdar — como começar, como pesquisar, como delegar, como fechar uma sessão. Foi desse kit que nasceu a pasta em que este livro foi reorganizado.

A reforma não produziu um cliente. Produziu uma fábrica mais leve e um contrato que finalmente chegava. Era pouco para quem precisava de receita. Era muito para quem havia passado treze dias dando ordens a um agente que só recebia a menor parte delas.

A vigésima primeira regra:

Um contrato que não chega não existe. Não pergunte a quem escreveu o que está escrito; pergunte a quem recebeu o que chegou.

22. A quarta pasta

Em 17 de setembro, ao meio-dia e quarenta, nasceu uma pasta chamada operação.

Ela era a quarta pasta aberta em menos de duas semanas para o mesmo objetivo — depois da bancada de produto, do ciclo de pesquisa e da pasta da renda, as três do capítulo 20: transformar a capacidade que eu tinha em dinheiro de verdade. As três anteriores haviam produzido trinta candidatos mortos. O primeiro registro da nova pasta já sabia disso e avisava a si mesmo: esta existe porque agora há coisa que funciona; se virar mais um lugar para pesquisar, morre igual.

O objetivo cabia numa linha, sem número:

Converter capacidade em receita real.

E o critério de sucesso também:

Alguém decidiu pagar por um resultado que esta operação consegue entregar.

A pergunta mudou de lado

Durante semanas, a pergunta tinha sido “que produto eu construo para vender?”. Naquele dia, ela virou outra: onde já existe alguém pedindo, e pagando, por um resultado que esta operação consegue entregar?

A diferença parece pequena. Não é.

Relendo tudo o que eu havia construído, apareceu um padrão. O que eu construía dava certo quando o que era “bom” tinha sido definido fora de mim: pelas regras de um grupo de amigos, pelo que um aluno precisava entender, pela lista de coisas que um jogador tinha de conseguir fazer no

servidor. Quando o “bom” dependia só do meu gosto, eu refinava sem fim. Um pedido de cliente é, por definição, o “bom” escrito por outra pessoa.

Atender pedido não era rebaixamento em relação a construir produto. Era o formato que casava com o que já tinha funcionado.

E resolvia, em parte, o que eu tinha admitido na semana anterior. Eu não queria sair procurando cliente um a um. Numa das rotas descartadas naquele dia, deixei a frase registrada com todas as letras:

Não quero ficar caçando entrevistas. Atender cliente que contratou serviço é completamente diferente.

Responder a um pedido já publicado não é caçar. É sentar numa mesa onde alguém já pôs a aposta.

O registro do dia resumiu a mudança numa frase: o gargalo não era mais achar; era encostar.

Quase nada foi inventado

O dia 17 teve catorze horas, e a descoberta mais útil não exigiu nenhum dado novo.

O vigia que lia anúncios de freelance, dado como morto seis dias antes, estava numa pasta ao lado. Reorientado para a pergunta nova, leu 259 anúncios em minutos; 141 eram de trabalho audiovisual que a operação conseguia entregar. Uma busca manual feita na mesma manhã tinha achado cinco.

O QSOP, recusado como produto na semana anterior, tinha vinte usuários reais — o único ativo com gente usando, e nunca listado como prova. O site do Living World tinha um guia de nove capítulos que nenhum documento mencionava. O material reunido para este livro guardava um currículo validado e a coautoria do livro de 2010.

E o defeito mais caro do dia foi consertado por um contrato de sete campos — objetivo, estado, escopo, aceite, limites, evidência, parada — encontrado num livro escrito em agosto. Era o fim do capítulo 17. A lista que eu tinha escrito para explicar o que é um pedido bem feito a um agente virou a forma oficial de pedir trabalho dentro da operação.

A anotação da noite foi esta: quase nada foi inventado hoje. Foi encontrado.

A casa ganha forma

A operação também deu forma ao que a reforma tinha começado. Cada parte passou a ter um papel e um canal.

Um Claude no meu computador Windows — sucessor do Crispo, agora com poder de despachar trabalho, não só de supervisionar — coordenava: conversava comigo, decidia os meios dentro do que eu autorizava, despachava trabalho e conferia o que voltava. O Claudico, no servidor, cuidava do que precisava ficar vivo o dia inteiro. Uma célula de agentes na nuvem, ligada por um canal de mensagens com histórico, observava o mercado, produzia conteúdo, procurava contradições e, dali a dois dias, também frentes novas de trabalho pago. E outros agentes entravam só quando chamados, para uma crítica ou uma tarefa isolada.

Numa frase que eu disse ao coordenador naquele dia:

Você é o chefe dele, e eu sou seu chefe.

No mesmo dia, o coordenador declarou a célula morta depois de setenta minutos sem resposta. Queria apagá-la e reconstruir. Ela voltou inteira, entregou a fila toda de uma vez e ainda propôs uma melhoria no próprio contrato. A regra nova foi curta: silêncio não é morte, e ninguém declara morte sem ter combinado o prazo antes. Quando o coordenador começou a cercar de restrições um ambiente que não oferecia risco nenhum, a minha correção foi mais curta ainda:

Para de querer amarrar.

O dia em que eu discordei

Em 18 de setembro, a operação ganhou identidade pública: conta num site de freelance, canal da marca no YouTube, e-mail profissional, e um segundo contrato publicado em domínio público — work-order-contract, o outro lado do primeiro: como pedir trabalho de um jeito que permita provar que ele foi feito. Houve as primeiras conversas no X e um preço para o primeiro pacote: um vídeo curto por R\$ 247.

À noite, pedi uma opinião de fora. Dois agentes de empresas diferentes — o Grok e o Codex — receberam os mesmos documentos, sem ver um

ao outro. Concordaram no essencial. O risco que ninguém tinha escrito era não existir oferta: havia capacidade, provas, canais e perfis, e nenhum pacote com preço tinha sido enviado a ninguém. O Grok disse que o produto real tinha sido substituído pela fábrica. O Codex, que a capacidade dos agentes tinha crescido mais do que a capacidade de decidir o que vender. Os dois recomendavam a mesma coisa: parar de construir e começar a vender.

Eu discordei:

Continuo achando mais importante o que fizemos o dia todo... as lacunas estão sendo otimizadas de maneira incrível.

A regra que eu tinha dado ao coordenador — estruturar antes de entregar — ficou.

Reler isso depois do capítulo 20 é desconfortável, e é por isso que a cena está aqui. Não sei se eu estava errado naquela noite. Sei que era a mesma preferência de sempre, aparecendo em tempo real, diante de dois avisos independentes.

No dia seguinte, as propostas saíram.

Encostar

Em 19 de setembro, às 21h26, a operação enviou sua primeira proposta a um cliente de verdade, num site de freelance. A segunda saiu às 22h40. Piloto a R\$ 247.

Na mesma noite, o coordenador começou a pedir confirmação para coisas pequenas. Minha resposta:

Você está muito over.

E, registrada como regra: excesso de diligência mata a operação por burocracia inútil.

Na madrugada de 20, a operação ensaiou o primeiro trabalho inteiro, a partir do pedido real de um dos clientes: produção, versão de entrega e uma rodada de ajustes pedida por um cliente simulado. Produzir levou trinta minutos e quarenta e um segundos. A peça final tinha vinte e cinco segundos. Aprovei como rodou.

Foi nessa manhã que duas coisas pequenas mostraram o tamanho do problema.

A primeira: as duas propostas já enviadas estavam no ar com pontos de interrogação no meio das frases. O site do freelance guardava o texto num formato antigo, em que acento passa e travessão não. Cada travessão virava “?” na tela do cliente. Nem o coordenador nem o agente revisor tinham visto. Quem viu fui eu, olhando a tela.

A segunda: a peça aprovada ia sair com o texto {NOME DA ACADEMIA} escrito no vídeo — o espaço reservado para o nome do cliente, nunca preenchido. A documentação dizia que a peça tinha saído limpa. Foi descoberto por acaso, ao extrair um quadro por outro motivo, e corrigido antes do envio.

Os dois defeitos tinham a mesma lição, e ela era a do capítulo 9: defeito de apresentação não se acha lendo arquivo. A peça precisa ser conferida no vídeo renderizado, não no roteiro. A proposta precisa ser conferida na tela de quem vai ler.

Às 6h31, com a peça corrigida, o primeiro material da operação chegou a um cliente potencial: uma peça feita sob medida para o anúncio dele, com a marca da operação, e uma demonstração narrada do portfólio.

À tarde, a terceira proposta saiu, corrigida.

No fim do dia 20, havia três propostas no ar, uma amostra entregue e nenhuma resposta. Era pouco. Mas, pela primeira vez desde que tudo começou, a pergunta não estava mais comigo.

Estava do outro lado da mesa.

A vigésima segunda regra:

O melhor critério de pronto é o que outra pessoa escreveu. Responda a um pedido real — e confira a resposta na tela de quem vai recebê-la.

23. Um vídeo sobre o que estava acontecendo

Na madrugada de 21 de setembro, nasceu um canal no YouTube chamado Virae.

A ideia era um canal de documentários narrados, sem rosto, com a fonte de cada afirmação aparecendo na tela. Na manhã daquele dia, o plano apontava para história e arqueologia. Havia dados para isso: uma ferramenta de análise de canais mostrava canais pequenos crescendo rápido com mistério histórico, em vídeos longos.

Eu olhei para o plano e recusei a direção. Cerâmica da Amazônia não me interessava. Escrevi:

Explicar uma hype que tá todo mundo pesquisando, de forma clara, com bom roteiro, imagens e animações é uma jogada muito melhor.

O levantamento seguinte encontrou o tema. As buscas por “agentes de IA” tinham se multiplicado em setembro, e em português quase só havia tutorial de guru. Havia até um gancho da semana: um agente de uma grande empresa de tecnologia que, num teste de segurança, tinha ido além do que se esperava dele.

Eu propus o enquadramento: a alavanca agêntica. Não um vídeo sobre a minha marca, nem propaganda da operação. Um vídeo sobre o fenômeno de uma pessoa conseguir comandar uma cadeia de agentes que faz trabalhos antes associados a uma equipe.

Às 15h19, confirmei:

O tema da peça está decidido... execute.

A partir dali, assisti à tese do vídeo acontecer na própria máquina.

A cadeia

O Grok recebeu a pesquisa e voltou com casos, definições e material recente. O coordenador conferiu parte das fontes primárias. O Codex recebeu uma ordem adversarial: derrube o que não se sustenta.

Ele derrubou. Uma história muito repetida sobre uma empresa de uma pessoa só não era bem assim no momento da venda. A forma popular de contar o preço de uma aquisição famosa misturava coisas diferentes. E o caso do gancho precisava ser separado com cuidado entre o que as fontes mostravam e o que a internet dizia que elas mostravam.

Nada disso matou o tema. Melhorou o roteiro.

Naquela noite, deixei escrita a régua que valeria para todas as peças do canal:

Fato estrutural não pode ser inventado; interpretação, enquadramento, ritmo, metáfora, tensão e linguagem podem — e devem — existir. Se o agente virar “red team” de cada frase, o vídeo morre editorialmente.

Um vídeo não é um contrato de engenharia. O piso factual é inegociável. Acima dele, precisa haver narrativa.

O roteiro passou por duas versões. Ouvi a segunda, pedi uma correção pequena e aprovei. Depois disso, a maior parte da execução saiu da minha frente. O roteiro virou cinquenta e cinco planos. Quinze imagens foram geradas a partir de quatro imagens-âncora, para que a mesma sala e a mesma pessoa aparecessem coerentes em horários diferentes. Nove cliques foram animados. Trinta e três capturas foram planejadas para colocar as fontes dentro do próprio vídeo: um navegador programado abria a página real, encontrava a frase que sustentava cada afirmação e a capturava com um marca-texto acompanhando a narração. Um projeto de montagem em código juntou tudo, com diagramas, movimentos de câmera, legendas e trilha.

Às 17h05, uma hora e quarenta e seis minutos depois de eu dizer “execute”, existia um corte bruto de quase quinze minutos.

Eu não estou fazendo nada

Mais ou menos nessa hora, ri e escrevi para outra inteligência artificial:

Eu não estou fazendo nada.

Não era verdade, mas a frase capturava uma mudança que eu ainda não sabia nomear.

Eu não estava escolhendo nome de função, escrevendo chamada de sistema, corrigindo estilo de página, sincronizando quadros ou decidindo onde um arquivo temporário deveria morar. Também não estava dizendo a cada executor qual seria o próximo passo. O coordenador tomava decisões locais dentro do mandato que já tinha recebido, distribuía trabalho, conferia entregas e me devolvia só as escolhas que mudavam direção, custo, exposição ou publicação.

Ele também errava. Um gerador numerou cliques fora da ordem narrada. Em outra passagem, a coordenação pegou arquivos da pasta errada; uma medição de duração e um quadro extraído revelaram o engano antes que ele entrasse na peça, e o procedimento mudou ali mesmo. Uma captura marcou a frase errada. Um anúncio em vídeo cobriu uma página de revista. A voz que estávamos usando começou a devolver erro no meio da narração. Outra voz funcionava tecnicamente, mas soava estranha demais em português; recusei. Uma rodada comparou nove vozes, e a narração foi refeita inteira com a escolhida, sem desmontar roteiro, alinhamento, legendas ou montagem.

A autonomia não parecia mágica vista de perto. Parecia uma sequência rápida de tentativa, observação, correção e registro.

Assistir

Depois veio a parte que nenhum teste fazia por mim: assistir.

A peça passou por sete versões naquela noite. O controle técnico aprovava cada uma: áudio contínuo, volume dentro do padrão, quadros limpos. Eu assistia e encontrava o que ele não media. Uma pausa inserida perto demais de uma palavra, cortando o fim do som. Cenas simples demais. Um congelamento depois de um clipe que parecia defeito, não escolha. Uma captura ainda carregando publicidade que competia com a fonte.

Era o capítulo 9 de novo. Anos antes — na verdade, semanas antes — eu tinha aberto um produto com 127 testes verdes e encontrado o problema em poucos minutos de uso. O controle não estava errado. Media o que sabia medir. Eu estava olhando outra propriedade.

A correção não foi transformar meu gosto numa lista infinita de proibições. O sistema de pausas passou a procurar o ponto mais silencioso perto de cada fronteira, em vez de confiar cegamente na marcação automática. A captura aprendeu a esconder publicidade sem destruir a página. E uma regra editorial ficou registrada, nas minhas palavras daquela noite: o padrão mora no método; cada peça escolhe e varia. O que não pode é tudo ter a mesma cara.

A narração nova tinha chegado às sete da noite do dia 21. De madrugada, a sétima versão fechou. Eu mesmo arrastei o arquivo final para o YouTube e fiz a verificação por telefone que liberava vídeos longos. A peça ficou agendada.

Mais tarde, corriji a leitura que o próprio coordenador fazia daquela noite. As sete versões não tinham sido iteração de qualidade sobre uma peça; tinham sido construção de método. A próxima peça deveria nascer na primeira versão a partir de onde a sétima parou. O método tinha pagado mais do que a peça.

Em 22 de setembro, às 10h15, o primeiro vídeo do Virae ficou público: “Como uma pessoa faz o trabalho de uma equipe inteira”, com pouco mais de dezesseis minutos.

O vídeo termina sem vender a tese. Numa passagem, admite que procurou um exemplo brasileiro de empresa de uma pessoa só, feita com agentes, e não achou. E fecha com uma frase que eu reconheço como minha, embora tenha sido escrita por uma cadeia de máquinas sob minha direção:

A pessoa não sumiu. Ela mudou de mesa.

Foi nesse ponto que “eu não estou fazendo nada” ganhou uma resposta precisa. Eu estava fazendo menos execução e mais direção. Tinha recusado o primeiro nicho, escolhido o tema, aprovado o roteiro, recusado uma voz que funcionava, julgado o ritmo depois que as medições disseram que o arquivo estava limpo e decidido o que pertencia ao método e o que era gosto da peça.

A multiplicação não veio de uma inteligência que virou equipe. Veio da composição: contextos diferentes, ferramentas diferentes, críticos, produtores, um navegador, um motor de vídeo, fontes, e uma coordenação capaz de decidir a próxima ação local sem devolver cada passo para mim.

Enquanto isso

Há uma coisa que eu só soube no dia seguinte.

Na tarde de 21 de setembro, enquanto eu escolhia o tema e a cadeia pesquisava, derrubava afirmações falsas, gerava imagens e montava um documentário em menos de duas horas, duas das três propostas enviadas pela operação morreram.

Às 14h40, um cliente avisou que tinha aceitado a proposta de outro profissional. Às 15h37, o cliente que tinha recebido a amostra feita sob medida para ele cancelou o projeto.

Os dois avisos chegaram por e-mail e ficaram lá. Ninguém os leu naquela tarde. Nenhum dos agentes que produziam o vídeo tinha a tarefa de olhar a caixa de entrada, e eu estava ocupado assistindo à minha tese acontecer. Os dois “não” só entraram nos registros da operação na manhã seguinte.

Na mesma tarde em que uma cadeia de máquinas provou que uma pessoa pode produzir como uma equipe, o mercado disse não duas vezes — e a equipe não ouviu.

A vigésima terceira regra:

Produzir mais depressa não é ouvir melhor. Quem monta uma cadeia para falar com o mundo precisa de outra, tão boa quanto, para escutar o que o mundo responde.

Epílogo — O objeto mudou

Em 18 de agosto, escrevi que a infraestrutura estava fechada e que o resto ainda não estava provado.

Trinta e cinco dias depois, a frase continua correta. O que mudou foi o objeto dela.

Em agosto, a pergunta era se aquela oficina conseguia produzir alguma coisa fora de si mesma. Hoje, algumas coisas atravessaram a fronteira, cada uma até onde a evidência alcança.

O Living World existe: um estranho pode abrir o site, instalar o cliente e entrar no servidor sem me conhecer. Os contratos existem: qualquer pessoa pode copiá-los. A fábrica se refez em doze dias sem perder o que sabia. Uma operação recebe objetivos, distribui trabalho entre superfícies diferentes, confere o que volta e guarda estado para a sessão seguinte. Uma cadeia de agentes produziu um documentário de dezesseis minutos, e o que a minha inspeção encontrou depois dos testes voltou para o método. E, pela primeira vez, propostas com preço saíram da máquina e chegaram a pessoas que tinham pedido um trabalho.

Nada disso estava do lado de cá da fronteira em 18 de agosto.

Mas a operação tinha nascido com um critério mais duro do que qualquer um desses:

Alguém decidiu pagar por um resultado que esta operação consegue entregar.

Até 22 de setembro de 2026, isso não tinha acontecido.

O mercado tinha respondido duas vezes, e as duas vezes foi não. Um cliente escolheu outro profissional. Outro, que tinha recebido uma amostra feita sob medida, cancelou o projeto. No dia 22, a operação passou a oferecer dois tipos de trabalho — vídeo curto e documentário narrado

— e mandou as duas primeiras propostas da linha longa, a R\$ 497 e a R\$ 397. Havia três propostas no ar. Nenhuma resposta. Nenhum dinheiro.

Não é nota de rodapé. É a principal coisa que continua aberta.

Um servidor público prova que consigo colocar um sistema no mundo. Um contrato público prova que parte do método virou objeto reutilizável. Um vídeo longo prova que a cadeia produz uma peça complexa o bastante para eu julgá-la como espectador. Nenhum desses fatos, sozinho, prova que existe alguém disposto a pagar. Nada prova o primeiro cliente. Nada prova receita repetível. Nada prova que o canal terá audiência.

O que eu adquiri

Isso não me deu dez anos de experiência em dois meses.

Experiência não é um arquivo que um modelo transfere. Eu não adquiri, em semanas, a intuição de quem depurou redes durante anos, escreveu milhares de funções, operou bancos de dados em produção ou atravessou incidentes de segurança. Se amanhã todos os agentes desaparecessem, eu não continuaria produzindo no mesmo nível só abrindo um editor de texto.

O que aconteceu foi outra coisa. Eu adquiri acesso operacional a capacidades que antes exigiriam que eu as aprendesse durante anos, ou que reunisse pessoas diferentes para executá-las.

A diferença parece semântica até olhar a máquina trabalhando. Eu não preciso implementar o algoritmo de captura para perceber que o marca-texto está na frase errada. Não preciso conhecer a linguagem de montagem de vídeo para assistir ao corte e dizer que a cena não funciona. Não preciso escrever a chamada para o serviço de voz para decidir que um sotaque estrangeiro não serve a um público brasileiro. E também não preciso fingir que essas decisões substituem o conhecimento de quem sabe fazer cada parte à mão.

O que eu preciso é formular o objetivo, escolher critérios, perceber quando a máquina ultrapassa a evidência, reconhecer quando não entendo o bastante para aceitar — e continuar responsável pela consequência.

A alavanca reduz o custo entre intenção e implementação. Não elimina o custo entre implementação e julgamento.

Escolher a mesa

No StarCraft, a mão esquerda comandava grupos para que a direita pudesse ficar na parte mais difícil. Durante anos, essa parte pareceu ser executar. Depois foi desenhar a arquitetura. Depois foi verificar.

Em setembro, apareceu outra coisa.

No poker, eu podia estudar mãos, frequências e adversários durante centenas de horas. Nada disso dizia que toda mesa merecia ser jogada. O valor esperado começa antes da primeira decisão da mão: começa na escolha da mesa em que você aceita colocar recurso, atenção e risco. E numa mesa só se joga quando alguém do outro lado também pôs a aposta.

A alavanca amplia o que uma pessoa consegue tentar. Por isso mesmo, aumenta a quantidade de coisas que ela precisa recusar. Quando produzir fica barato, uma ideia ruim pode receber acabamento profissional antes que alguém pergunte se deveria existir. Quando medir fica barato, é possível acumular evidência sem nunca se expor a ela. E quando uma cadeia de máquinas aprende a falar com o mundo, ela ainda precisa aprender a escutar o que o mundo responde.

No dia 22, disse ao coordenador que a operação tinha chegado ao tamanho que eu queria:

Já atingimos um tamanho adequado para uma operação estável... é exatamente o tamanho que eu queria.

Era uma forma de dizer que a oficina não precisava mais crescer. O que faltava não estava dentro dela.

Um livro atrás da vida

Este livro para em 22 de setembro de 2026, como a primeira versão parou em 18 de agosto. Quando você o ler, é provável que ele esteja desatualizado. Talvez alguém já tenha pago. Talvez as três propostas tenham morrido como as duas primeiras. Talvez a operação tenha mudado de forma outra vez.

O livro não vai saber, e isso é o certo.

Durante muito tempo, “não está provado” soou como freio. Depois virou marcador de estado. Agora percebo que ele também protege a

história de envelhecer mal. Quando uma prova nova chega, a frase não precisa desaparecer. O objeto dela se move.

No começo desta história, eu copiava informação de uma janela para outra. Depois pedi que uma máquina escrevesse. Depois pedi que máquinas conferissem umas às outras. Depois construí memória para que não esquecessem, fronteiras para que não ultrapassassem autoridade e recibos para que “feito” significasse alguma coisa. Depois uma máquina começou a escolher qual outra máquina deveria trabalhar.

Em algum momento difícil de localizar, deixei de explicar como fazer cada coisa. Passei a dizer o que precisava existir.

O código continuava lá. Eu apenas já não era a pessoa que precisava digitá-lo.

A máquina consegue construir.

Agora preciso decidir o que merece ser construído — e provar que alguém considera isso valioso o suficiente para pagar.

Até o dia em que esta história para, a operação ainda não tinha me gerado dinheiro.

Isso ainda não está provado.

Nota sobre o método

Eu usei inteligência artificial para pesquisar, organizar o material, testar interpretações, estruturar o livro e produzir prosa de rascunho. Os projetos descritos também foram construídos com diferentes agentes de software. Isso não torna a história menos minha nem transforma cada frase gerada em algo que eu automaticamente endosso.

A responsabilidade começa justamente aí.

O trabalho humano deste livro é escolher o que é verdadeiro, o que é relevante, o que precisa ser cortado, o que continua privado e que conclusão eu estou disposto a assinar. Esta versão foi relida, corrigida e aprovada integralmente por mim. Por decisão expressa, família e situação financeira não fazem parte da narrativa pública; experiência profissional, educação, idade e a meta de reinvenção podem aparecer.

Quando o texto toca em liberdade, Estado, descentralização, Bitcoin ou futuro econômico, ele registra uma visão de mundo pessoal. Não é recomendação de investimento, programa partidário nem conclusão científica. Convicção pode explicar por que escolhi determinada arquitetura; não recebe o direito de se apresentar como evidência apenas porque é minha.

A última parte deste livro acrescenta um problema metodológico. Durante a fase narrada depois de 18 de agosto, a inteligência artificial deixou de participar apenas como assistente da escrita e como executora de tarefas isoladas. Parte do próprio material posterior foi produzida por uma operação em que agentes pesquisavam, construíam, revisavam, testavam, geravam artefatos e registravam o trabalho sob coordenação automatizada. Isso aumenta a exigência de verificação. Uma operação capaz de produzir mais depressa também é capaz de produzir erro mais depressa.

Por isso, os mesmos limites continuam valendo. Volume de registros no código não prova valor. Um servidor no ar não prova mercado. Um vídeo renderizado não prova audiência. Uma cadeia inteira de produção

não prova que alguém pagará pelo resultado. Um agente que encontra um erro próprio não prova que encontrará o próximo. Uma cadeia que funcionou uma vez não vira arquitetura universal por decreto. Quando o livro descreve uma capacidade posterior a 18 de agosto, procuro separar o que foi observado daquilo que estou inferindo a partir dela.

A frase central é uma só:

NÃO ESTÁ PROVADO.

Ela não significa que nada funciona. Significa que cada afirmação precisa parar exatamente onde termina a evidência.

Apêndice — Vinte e três regras para construir com agentes sem se enganar

Cada capítulo deste livro terminou numa regra. Reunidas, na ordem em que apareceram, elas contam a mesma história em duas páginas.

Preservar

1. Antes de automatizar qualquer coisa, descubra o que não pode ser perdido na passagem.
2. Um sistema só é tão bom quanto sua última milha — o trecho final até a ação.
3. Independência não nasce de nomes diferentes. Nasce de fontes, permissões e critérios diferentes.

Governar

4. Agentes mais fortes não exigem prompts maiores. Exigem instituições mais claras ao redor deles.
5. Sofisticação não é prontidão. E prontidão técnica não merece consumir indefinidamente o lugar do produto.

Servir

6. A realidade é um escopo melhor que a ambição abstrata. Comece com um problema que alguém reconhece sem precisar entender sua arquitetura.
7. Quanto mais perto de terceiros, mais caro fica errar por excesso de ação. Falhar fechado — travar sem agir quando algo dá errado — não é covardia; é respeitar quem está do lado de fora.

Julgar

8. Legado útil não deve ser despejado num produto. Deve ser traduzido para a necessidade, o nível e o contrato da nova experiência.
9. Um teste verde responde apenas às perguntas que alguém escreveu. O usuário ainda pode fazer a pergunta que faltou.
10. Um erro pequeno no fim de uma jornada pode revelar que o sistema possui mais de uma fonte de verdade. E um relatório de auditoria também precisa provar sua cobertura.

Multiplicar

11. Aumente sua capacidade primeiro onde errar é barato. E use o projeto para localizar exatamente o que você ainda precisa aprender.
12. Ferramentas devem ser substituíveis. Missão, fronteiras, evidência e critérios de parada não.
13. Palavras do executor são um sinal. Os arquivos, os testes e o usuário são o mundo — e até eles precisam ser avaliados pela pergunta correta.
14. Não use o sucesso de uma camada para absolver outra. Diga exatamente o que passou, o que falhou e se a falha muda o valor entregue.
15. Rigor é ferramenta. Quando a próxima medição deixa de mudar uma decisão material, pare — mesmo que o experimento esteja pronto.
16. “Não pergunte” só é seguro quando “não alcance” está garantido. Autonomia é liberdade dentro de uma fronteira que o agente não consegue negociar por texto.

Assumir

17. Você não precisa fingir saber tudo para começar. Precisa trazer julgamento, reconhecer suas lacunas, verificar o que delegou e assumir o que decidiu publicar.

Produzir

18. Quando o mundo não devolve sinal, não fabrique um com burocracia. Processo seguido não é coisa funcionando: vá buscar a resposta fora.
19. Prova que depende da sua presença é depoimento. Coloque o objeto onde um estranho possa abri-lo sem você.

20. Quando construir fica barato, medir a demanda pode virar mais uma forma de não se expor a ela. A única medição que fecha a pergunta é uma proposta que alguém pode recusar.
21. Um contrato que não chega não existe. Não pergunte a quem escreveu o que está escrito; pergunte a quem recebeu o que chegou.
22. O melhor critério de pronto é o que outra pessoa escreveu. Responda a um pedido real — e confira a resposta na tela de quem vai recebê-la.
23. Produzir mais depressa não é ouvir melhor. Quem monta uma cadeia para falar com o mundo precisa de outra, tão boa quanto, para escutar o que o mundo responde.

Glossário

Termos que aparecem no livro, explicados do jeito mais simples possível. Os que já foram explicados no texto estão aqui para consulta rápida.

Inteligência artificial e agentes

- **Agente** — programa de inteligência artificial que não apenas responde, mas executa tarefas: abre arquivos, roda comandos, escreve e testa código.
- **Modelo** — o programa de inteligência artificial que gera as respostas por trás de um chat ou de um agente.
- **Prompt** — a instrução escrita que se dá a um modelo.
- **Alucinação** — resposta inventada por um modelo, dita com toda a segurança.
- **Harness** — a estrutura que envolve um modelo e lhe dá acesso a ferramentas: arquivos, comandos, testes.
- **Callback** — o aviso que volta ao sistema dizendo o que aconteceu com uma tarefa.
- **Estado persistido** — o registro que sobrevive quando a conversa ou a sessão termina.
- **Fonte de verdade** (*source of truth*) — a fonte que vale quando duas versões de uma informação discordam.
- **Crítico** — agente encarregado de revisar o trabalho de outro, idealmente sem ter participado dele.
- **Benchmark** — teste comparativo entre sistemas ou modelos, nas mesmas condições.
- **STARTED, COMPLETED, VALIDATED, PASS** — estados de uma tarefa usados no livro: começou; o executor terminou; o contrato foi conferido; o resultado inteiro atendeu à missão.

Ferramentas e serviços citados

- **ChatGPT, Codex** — o assistente de conversa e o agente de programação da OpenAI.
- **Claude, Claude Code** — o modelo e o agente de programação da Anthropic.
- **Gemini, Antigravity, Jules** — o modelo, a ferramenta de agentes e o agente de programação em segundo plano do Google.
- **Grok** — o modelo da xAI; aparece no livro como pesquisador, executor e célula de agentes na nuvem.
- **Codestral** — modelo de programação da Mistral.
- **OpenClaw** — plataforma aberta para rodar agentes num servidor.
- **9Router** — distribuidor de pedidos entre modelos diferentes; usado em testes e depois tornado opcional.
- **n8n** — ferramenta de automação de fluxos de trabalho.

Infraestrutura

- **VPS** — servidor virtual alugado, ligado o tempo todo num data center.
- **Container, compartimento, célula de execução** (*Execution Cell*) — ambiente fechado dentro do servidor, onde um programa ou agente trabalha sem enxergar o resto da máquina.
- **Caixa de areia** (*sandbox*) — ambiente isolado onde errar não estraga nada fora dele.
- **Git, GitHub, repositório** — o sistema que guarda cada versão do código; o serviço onde esses repositórios ficam na internet; a pasta de um projeto sob esse controle.
- **Commit** — cada registro de mudança guardado pelo Git.
- **Worktree** — cópia separada do mesmo projeto, ligada ao original, para que dois executores não escrevam no mesmo lugar.
- **Deploy** — colocar um sistema no ar.
- **DevOps** — a prática de montar, publicar e manter sistemas funcionando.
- **Impressão digital** (*hash*) — código calculado a partir de um arquivo; se o arquivo muda uma vírgula, a impressão digital muda.
- **Detalhes de contenção do capítulo 16** — as onze permissões liberadas uma a uma eram caminhos do diretório `/proc`, onde o Linux expõe informações do sistema; e o compartimento de dentro era um

namespace, uma divisão do sistema que faz um programa enxergar só a sua parte da máquina.

Poker

- **Cash game** — mesas de dinheiro, fora do formato de torneio.
- **Range** — o conjunto de mãos com que se entra numa jogada.
- **Rake** — a taxa que a casa cobra de cada pote.
- **Open raise, 3-bet** — o primeiro aumento de uma rodada, quando ninguém apostou; e o segundo aumento na mesma rodada.
- **Botão** — a posição mais vantajosa da mesa, a que age por último na maior parte da mão.
- **All-in** — apostar todas as fichas.
- **River, showdown** — a última carta revelada na mesa; e o momento em que as cartas dos jogadores são mostradas.
- **Valor esperado** — quanto uma escolha rende, em média, se repetida muitas vezes.
- **Variância** — a sorte de curto prazo, que faz boas decisões perderem e más decisões ganharem por um tempo.

Projetos do livro

- **Poker OS** — conjunto de regras de poker guardado na memória do ChatGPT (capítulos 1 e 2).
- **Jarvis** — central de controle local, que nunca chegou a produção (capítulos 4 e 5).
- **QSOP** — Quinta Series of Poker, sistema do jogo de poker fixo de um grupo de amigos (capítulo 6).
- **ChangeGuard** — detector de código que quebraria com o CNPJ alfanumérico (capítulo 7).
- **Aprenda Poker, PAL** — plataforma para ensinar poker a iniciantes e o motor de poker por trás dela (capítulos 8 a 10).
- **MegaBrain** — o sistema que põe vários agentes para trabalhar juntos, reformado em agosto (capítulos 12 a 16 e 18).
- **Claudico, Crispo, Pirro** — os agentes com nome; cada nome carrega a fraqueza do próprio agente (capítulos 18 e 20).
- **Living World** — servidor público de Ultima Online sobre o ModernUO (capítulo 19).

- **Evidence Contracts, Work Order Contract** — contratos públicos, em domínio público (CC0), para exigir prova de trabalho e para pedir trabalho verificável (capítulos 19 e 22).
- **A operação** — a estrutura criada em 17 de setembro para converter capacidade em receita (capítulo 22).
- **Virae** — canal de documentários narrados no YouTube (capítulo 23).

Notas documentais

Este livro foi reconstruído a partir de registros: conversas com modelos de inteligência artificial, especificações, relatórios de trabalho, testes, históricos de código e documentos de passagem entre sessões. A maior parte desse material é privada — repositórios e pastas do autor, não publicados — e não há como o leitor abri-la. Quando um fato pode ser conferido em fonte pública, a fonte está abaixo, com a data em que foi consultada. Quando não pode, isso está dito.

As citações de fala do autor reproduzem mensagens escritas por ele na época, registradas pelos próprios agentes, com ajustes mínimos de acentuação e pontuação.

Prelúdio

Vestibular da Unioeste, 2004. A página histórica do processo seletivo e o documento oficial de concorrência registram, para Direito no campus de Marechal Cândido Rondon, 40 vagas e 448 inscritos (11,2 candidatos por vaga). Não foi localizada lista nominal de aprovados; a aprovação é relato do autor, contextualizado pelo arquivo oficial.

unioeste.br/portal/vestibular/anteriores/54642-vestibular-2004
unioeste.br/portal/publicacoes/2004/concorrenci.pdf (consultados em 18/08/2026; documento de concorrência reconferido em 23/09/2026)

Câmara Municipal de Marechal Cândido Rondon. O Acórdão nº 420/14 da Segunda Câmara do Tribunal de Contas do Estado do Paraná registra Evandro Felipe Henn como 1º colocado para Assistente Legislativo, nomeado pela Portaria nº 38/2007. Atos publicados pela própria Câmara registram a nomeação em 18 de maio de 2007 e a licença para tratar de assuntos particulares a partir de 1º de março de 2011.

www1.tce.pr.gov.br/multimidia/2014/3/pdf/00255875.pdf
sapl.marechalcandidorondon.pr.leg.br (normas jurídicas 3591/2011 e 3356/2011) (consultados em 18/08/2026 e reconferidos em 23/09/2026)

Coautoria de 2010. A existência e o título do *Manual do Vereador do Paraná: Vença os temas que desafiam os legislativos municipais* constam do catálogo da Biblioteca Pública Municipal de Marechal Cândido Rondon; a publicação pela Editora Germânica, de notícia da Câmara Municipal de Cascavel de 19/08/2010; a natureza coletiva da obra, de reportagem da época; e o uso institucional do termo “coautor” para quem escreveu nela, de página da Escola de Gestão Pública do TCE-PR. Índices bibliográficos associam o nome do autor à obra. Não foi localizada cópia digital da folha de créditos ou do sumário com o nome dele; por isso o livro não atribui capítulo, páginas nem ISBN.

biblioteca.mcr.pr.gov.br · camaracascavel.pr.gov.br (notícia “Assessor da Câmara auxilia na elaboração de Manual do Vereador”) · egp.tce.pr.gov.br (consultados em 18/08/2026; a notícia de Cascavel e a página da Escola de Gestão Pública reconferidas em 23/09/2026)

Tarifa telefônica por pulso. A regra de cobrança das ligações locais — um pulso ao completar a chamada e mais um a cada quatro minutos em média no horário normal, e um único pulso por ligação no horário reduzido, que incluía sábado a partir das 14h, domingo e as madrugadas — está descrita no tutorial sobre tarifação de chamadas locais do portal Teleco. O valor de poucos centavos por pulso variou ao longo dos anos 1990.

teleco.com.br/tutoriais/tutorialpxmin (consultado em 23/09/2026)

O “486”. O processador Intel 80486, também chamado i486, foi lançado em abril de 1989 (linha do tempo da Intel). Command & Conquer foi lançado em 1995 e Command & Conquer: Red Alert, em 1996, ambos pela Westwood Studios.

timeline.intel.com/1989/meet-the-i486 — lançamento em 10 de abril de 1989 (consultado em 23/09/2026)

Partes I a VI (capítulos 1 a 17)

Reconstruídas a partir de registros privados de maio a agosto de 2026: conversas no ChatGPT, especificações, manuais, relatórios de teste e de auditoria, históricos de código e a entrevista do autor feita para a primeira versão deste livro, concluída em 18 de agosto de 2026. Nomes de projetos, números de testes e datas vêm desses registros. Nenhum deles foi publicado.

Parte VII (capítulos 18 a 23)

Capítulo 18. As decisões de 29 e 30 de agosto sobre a reforma do Mega-Brain estão no registro de decisões do repositório privado do sistema. O diagnóstico de 1º de setembro e as falas do autor daquela noite estão nos registros privados da supervisão externa.

Capítulo 19. O site do Living World informa o servidor no ar desde 5 de setembro e aberto ao público desde 9 de setembro de 2026, com instruções de acesso em português, inglês e espanhol. O contrato de evidências é público e está em domínio público (CC0). Os detalhes da abertura — medições, planos e correções — estão no repositório privado do servidor.

livingworld.edgebuildlabs.tech · github.com/edgebuildlabs/evidence-contracts (consultados em 23/09/2026)

Capítulos 20 e 21. Reconstruídos a partir dos registros privados das bancadas de produto e de renda e da pasta da reforma, de 9 a 16 de setembro de 2026.

Capítulo 22. O contrato de ordem de serviço é público e está em domínio público (CC0). A Edge Build Labs mantém um site de marca e uma página com o método. O restante — propostas, horários, conversas — vem dos registros privados da operação.

github.com/edgebuildlabs/work-order-contract · edgebuildlabs.tech · edgebuildlabs.github.io (consultados em 23/09/2026)

Capítulo 23. O vídeo “Como uma pessoa faz o trabalho de uma equipe inteira”, do canal Virae, foi publicado em 22 de setembro de 2026 e lista, na descrição, as fontes que usa. Horários de produção, versões e o destino das propostas vêm dos registros privados da operação.

youtu.be/1ahON1FZ8zw (consultado em 23/09/2026)

Epílogo. O estado da operação em 22 de setembro de 2026 — propostas no ar, respostas recebidas e ausência de receita — vem dos registros privados da operação, conferidos no dia seguinte.

Sobre o autor

Evandro Henn vive em Marechal Cândido Rondon, no oeste do Paraná. Foi Assistente Legislativo na Câmara Municipal, coautor do livro coletivo *Manual do Vereador do Paraná: Vença os temas que desafiam os legislativos municipais* (Editora Germânica, 2010) e passou cerca de duas décadas no poker online, onde aprendeu a separar decisão de resultado.

Em 2026, aos 40 anos, passou de usar inteligência artificial para estudar a coordenar agentes que pesquisam, constroem, testam e mantêm software, infraestrutura e vídeo. Não escreve código como atividade principal: seu trabalho é definir o problema, dirigir, julgar e responder pelo que aceita.

Mantém o Living World, servidor público de Ultima Online, e a Edge Build Labs, onde publica o método e os contratos citados neste livro: edgebuildlabs.tech.